

UNIVERSITATEA BABEȘ-BOLYAI
Facultatea de Științe Economice și Gestiunea Afacerilor
Specializarea Informatică Economică

Lucrare de licență

Absolvent,
Mălina-Ioana **CILIBIA**

Coordonator științific,
Lect. univ. dr. Cristina **OSMAN**

2025

UNIVERSITATEA BABEȘ-BOLYAI
Facultatea de Științe Economice și Gestiunea Afacerilor
Specializarea Informatică Economică

Lucrare de licență

Aplicație web în Laravel pentru adopția pisicilor, cu
suport AI și funcționalități e-commerce

Absolvent,
Mălina-Ioana **CILIBIA**

Coordonator științific,
Lector univ. dr. Cristina **OSMAN**

2025

Rezumat

Lucrarea de față urmărește realizarea unei aplicații web care să simplifice și să încurajeze adopția pisicilor, răspunzând unei nevoi reale din România. Am construit o aplicație unde iubitorii de animale pot descoperi pisici disponibile, pot trimite cereri de adopție, programa vizite și împărtăși povești despre experiența lor. Pentru a asigura interactivitatea am implementat notificări în timp real, donații online și un modul AI care recunoaște rasa pisicii dintr-o imagine. Totul este gestionat printr-un panou dedicat administratorilor, care oferă control complet asupra conținutului și statisticilor. Ideea a pornit din pasiunea mea pentru pisici și dorința de a face procesul de adopție mai simplu și eficient. Rezultatul final este o aplicație ghidată de nevoile reale ale utilizatorilor și de dorința de a contribui la o schimbare pozitivă.

Cuprins

Lista tabelelor și figurilor.....	iv
Introducere.....	1
1. Metodologie	2
1.1 Procesul de dezvoltare al aplicației.....	2
1.2 Tehnologii specifice.....	3
1.2.1 Tehnologii	3
1.2.2 Framework-uri.....	3
1.2.3 Librării.....	4
2. Analiza sistemului informatic	5
2.1 Motivația dezvoltării.....	5
2.2 Analiza contextului	5
2.3 Colectarea datelor	6
2.4 Obiectivele	7
2.5 Tipuri de utilizatori și nevoile acestora.....	8
2.6 Analiza proceselor	10
2.6.1 Procesul As-Is	10
2.6.2 Procesul To-Be.....	10
3. Proiectarea sistemului informatic.....	12
3.1 Proiectarea bazei de date.....	12
3.2 Arhitectura generală a sistemului.....	13
3.3 Modelarea comportamentului aplicației.....	16
3.4 Proiectarea fluxurilor de date	20
3.5 Infrastructura de implementare	21
4. Implementarea	22
4.1 Componente funcționale ale aplicației.....	22
4.1.1 Back-End.....	22
4.1.2 Front-End.....	23
4.2 Interfața utilizatorilor	25
4.2.1 Cerere de adopție.....	25
4.2.2 Programări	27
4.2.3 Donații	31
4.2.4 Povești de adopție	35
4.3 Interfața administratorilor	37
4.3.1 Statistici.....	37
4.3.2 Gestionarea cererilor și a programărilor.....	38
4.3.3 Notificări	42
4.4 Inteligență artificială	45
4.4.1 Fundamente teoretice.....	45
4.4.2 Implementarea componentelor AI în aplicație.....	46
4.4.3 Pregătirea setului de date	47
4.4.4 Antrenarea și evaluarea modelului	50
4.4.5 Integrarea modelului în aplicație	53
5. Evaluarea	55
5.1 Testarea manuală	55
5.2 Testarea automată	57
Concluzii	60
Bibliografie	61
Anexe.....	63

Lista tabelelor și figurilor

Tabele:

Tabel 1. Rezultatele obținute în timpul antrenării și validării modelului AI.....	52
--	----

Figuri:

Figură 1. Diagrama Fishbone.....	6
Figură 2. Grafic Pareto.....	7
Figura 3. Diagrama Use Case a aplicației.....	9
Figură 4. Diagrama BPMN As-Is	11
Figură 5. Diagrama BPMN To-Be.....	11
Figură 6. Diagrama entitate-relație (ERD).....	13
Figură 7. Diagrama de componente pe partea de utilizator	14
Figură 8. Diagrama de componente pe partea de admin.....	15
Figură 9. Diagrama de activități pentru procesul de adopție	16
Figură 10. Diagrama de activități pentru procesul de povești de adopție	17
Figură 11. Diagrama de stări.....	18
Figură 12. Diagrama de secvență.....	19
Figură 13. Diagrama fluxurilor de date (DFD).....	20
Figură 14. Diagrama de deployment.....	21
Figură 15. Exemple de imagini din Oxford-IIIT Pet Dataset [23]	47
Figură 16. Numărul de imagini per clasă în setul de date.....	48
Figură 17. Compararea distribuției imaginilor între seturile de antrenament și test.....	50
Figură 18. Evoluția pierderii și acurateței modelului în timpul antrenării	53
Figură 19. Cazul de testare manuală 1	55
Figură 20. Cazul de testare manuală 2	56
Figură 21. Cazul de testare manuală 3	56
Figură 22. Cazul de testare automată 1	57
Figură 23. Cazul de testare automată 2	58
Figură 24. Cazul de testare automată 3	58

Introducere

Această lucrare a pornit în principal de la pasiunea mea pentru pisici și din dorința mea de a contribui la o schimbare reală în acest sens. În România, adopția animalelor este încă un proces complicat și de multe ori greoi atât pentru cei care vor să adopte, cât și pentru centrele care încearcă să găsească familii pentru animale. De aici a venit ideea mea de a crea o aplicație web care să simplifice și digitalizeze tot acest proces. Aplicația realizată de mine este menită să încurajeze adopția ca soluție principală pentru reducerea numărului de pisici fără stăpân.

Scopul principal al aplicației este de a crea un spațiu prietenos, în care iubitorii de pisici să poată descoperi pisici disponibile, să trimită cereri ușor, să programeze vizite fără bătăi de cap și să fie ținuti la curent prin notificări în timp real. În același timp, am ținut cont de nevoile celor care administrează centrele de adopție, mai exact să poată gestiona cererile eficient, să primească sprijin prin donații și să ofere informații corecte, inclusiv prin recunoașterea rasei unei pisici cu ajutorul inteligenței artificiale. Aplicația include funcționalități precum formular digital de adopție, calendar interactiv pentru programări, secțiune de povești, sistem de donații, notificări live și un panou de administrare complet.

Pe parcursul lucrării, am ales să lucrez într-un mod organizat, inspirat de principiile metodologiei Scrum, pe care le-am adaptat la un proiect individual. Am împărțit dezvoltarea aplicației în etape clare, stabilindu-mi obiective săptămânale pentru fiecare funcționalitate majoră. Pentru partea de implementare, am folosit limbajul PHP împreună cu framework-ul Laravel, o alegere foarte potrivită pentru obiectivele mele. Laravel mi-a oferit o structură logică, ușor de gestionat, dar și instrumente moderne de dezvoltare.

Lucrarea mea este structurată în mai multe capitole. Capitolul 1 descrie metodologia de lucru și justificarea tehnologiilor utilizate. Capitolul 2 analizează contextul social și funcțional al aplicației, incluzând nevoile utilizatorilor, procesele și motivația personală. Capitolul 3 descrie proiectarea sistemului informatic, prezentând baza de date, arhitectura aplicației și comportamentul acesteia prin diagrame UML și BPMN. Capitolul 4 detaliază procesul de implementare, evidențiind principalele componente funcționale și tehnice. Capitolul 5 reprezintă partea de testare efectivă a aplicației.

1. Metodologie

Capitolul următor reprezintă metodologia de lucru folosită pentru a realiza aplicația din cadrul acestei lucrări. În acest capitol, am detaliat etapele procesului de dezvoltare, selecția tehnologiilor specifice și justificarea alegerii acestora.

1.1 Procesul de dezvoltare al aplicației

Pentru organizarea procesului de dezvoltare al aplicației prezentate în această lucrare, am urmat o variantă adaptată a metodologiei Scrum, potrivită pentru proiecte individuale. Scrum este o metodă de gestionare a proiectelor caracteristică programării agilă, care constă în lucrul în etape mici, cu experimentări continue și feedback regulat, care permite învățarea și îmbunătățirea continuă pe parcursul dezvoltării.

Unul dintre principiile acestei metodologii este „volatilitatea cerințelor”, prin care se recunoaște că, pe parcursul realizării unui proiect, nevoile și preferințele clienților se pot schimba. Astfel de situații neprevăzute sunt dificil de gestionat prin metode tradiționale, ceea ce transformă flexibilitatea Scrum într-un avantaj important. Această abordare este una empirică, pornind de la ideea că problema nu poate fi definită complet de la început, ci necesită adaptare continuă la cerințele care apar, la evoluția tehnologică și la schimbările din piață [1].

Primul pas în realizarea aplicației a fost cercetarea. Am analizat mai multe site-uri de adopție de pe internet și m-am documentat mai mult despre ce înseamnă un proces de adopție al unei pisici, punând accent pe ceea ce lipsește în prezent. În urma acestor analize, mi-am făcut o listă clară în Word atât cu obiectivele mele pentru această aplicație cât și cu funcționalitățile principale pe care voiam să le aibă.

După ce mi-am stabilit tot ce aveam de făcut, mi-am ales și niște termene limită pentru toate acestea. M-am hotărât astfel să dedic câte o săptămână pentru fiecare funcționalitate principală precum cererile de adopție, programările, notificările, sistemul AI sau donațiile, stabilindu-mi obiective clare. Finalizarea fiecărei funcționalități am realizat-o prin testare. Această abordare, cu prioritizare clară, m-a ajutat să mențin controlul asupra progresului și să gestionez complexitatea aplicației.

1.2 Tehnologii specifice

Pentru a realiza aplicația din această lucrare, am căutat care ar fi tehnologiile, framework-urile și librăriile cele mai potrivite. Am realizat selecția acestora în funcție de flexibilitate, performanță și compatibilitatea cu cerințele stabilite de mine anterior. În continuare este o trecere în revistă a instrumentelor utilizate.

1.2.1 Tehnologii

PHP este limbajul de programare pe partea de server pe care am ales să îl folosesc pentru aplicația prezentată datorită arhitecturii sale clare și integrării native cu Laravel.

HTML (Hypertext Markup Language) este limbajul standard utilizat pentru structurarea conținutului unei pagini web. Am ales să folosesc HTML în dezvoltarea aplicației deoarece oferă o soluție simplă și eficientă pentru definirea componentelor multimedia.

CSS (Cascading Style Sheets) este limbajul utilizat pentru stilizarea documentelor HTML, definind aspectul vizual al elementelor afișate într-un browser [2]. Am ales CSS pentru că oferă control complet asupra prezentării conținutului web și permite separarea clară dintre structură și design. Pentru stilizare mai eficientă, am utilizat și framework-ul Bootstrap, o soluție open-source creată pentru a standardiza instrumentele front-end [3].

JavaScript este un limbaj de programare folosit pentru a adăuga interactivitate și comportamente dinamice în paginile web. Am ales JavaScript deoarece permite construirea unor interfețe moderne, care reacționează instant la acțiunile utilizatorului, fără a necesita reîncărcarea paginii [4].

1.2.2 Framework-uri

Pentru dezvoltarea aplicației, am folosit framework-ul Laravel, deoarece este o soluție modernă, sigură și eficientă pentru construirea aplicațiilor web. Laravel se bazează pe arhitectura MVC, unde modelul gestionează datele prin Eloquent ORM, view-ul (Blade) afișează conținutul, iar controllerul leagă logica aplicației de interfață [5].

Vue.js este un framework destinat construirii de interfețe interactive. Se bazează atât pe JavaScript cât și pe CSS și HTML pentru a oferi un model declarativ bazat pe componente, eficientizând dezvoltarea de interfețe de orice complexitate. Am utilizat Vue.js pentru că oferă o structură clară bazată pe componente, fiind ideal pentru interfețele dinamice și reactive din aplicația mea [6].

1.2.3 *Librării*

jQuery este o librărie al cărei scop este de a facilita utilizarea JavaScript pe o aplicație web. Acesta simplifică manipularea documentului HTML, încapsulând o mulțime de sarcini comune care necesită multe linii de cod JavaScript. Am ales să folosesc jQuery pentru a simplifica manipulările dinamice ale DOM-ului și integrarea rapidă a unor funcționalități interactive.

FullCalendar.js este o bibliotecă JavaScript care permite afișarea și gestionarea evenimentelor într-un calendar interactiv. L-am folosit pentru a permite afișarea interactivă a unui calendar pentru programări.

SweetAlert este o bibliotecă care oferă ferestre modale personalizate și stilizate pentru alerte. Oferă alerte modale elegante și ușor de personalizat.

Toastr.js este o librărie JavaScript pentru afișarea notificărilor de tip „toast”. L-am folosit pentru afișarea de notificări fără a întrerupe fluxul aplicației.

Stripe.js este o bibliotecă JavaScript oferită de Stripe pentru integrarea rapidă și sigură a plăților online. L-am ales pentru integrarea plăților online.

Pusher.js este o librărie pentru comunicarea în timp real între server și client, bazată pe WebSockets. Este esențială pentru o experiență interactivă a utilizatorilor în aplicația mea.

TensorFlow.js este o bibliotecă dezvoltată de Google care permite rularea modelelor de inteligență artificială direct în browser, folosind JavaScript. L-am folosit pentru a permite rularea modelelor de inteligență artificială direct în browser, fără a necesita procesare pe server.

2. Analiza sistemului informatic

În cadrul acestui capitol, mi-am propus să analizez contextul și necesitatea dezvoltării unei aplicații web pentru adopția pisicilor. Vor fi evidențiate categoriile de utilizatori, funcționalitățile principale ale platformei și beneficiile oferite. Scopul analizei este clarificarea nevoilor aplicației înainte de proiectarea tehnică.

2.1 Motivația dezvoltării

Motivația dezvoltării aplicației provine atât din dorința de a contribui la o soluție pentru găsirea unui cămin pentru fiecare pisica cât și dintr-o pasiune personală pentru acest subiect. Am fost întotdeauna pasionată de animale, în special de pisici, și am avut norocul să cresc alături de câteva pisici care mi-au adus multă bucurie pe parcursul anilor.

Motivația principală a fost totodată, impactul social pe care îl poate avea această aplicație. Pisicile fără adăpost reprezintă o problemă majoră peste tot în lume și mai ales în Europa, cu sute de mii de pisici abandonate în fiecare an. Programele de adopție, pot contribui semnificativ la reducerea acestui fenomen atunci când sunt implementate eficient. Potrivit celor mai recente date, în Europa trăiesc aproximativ 113 milioane de pisici domestice. În paralel, estimările realizate de diverse organizații pentru protecția animalelor arată că, anual, sute de mii de pisici sunt abandonate pe teritoriul european. Într-un context în care numărul animalelor fără adăpost este în continuă creștere, am simțit nevoia de a contribui la o soluție care să încurajeze adopția și să promoveze responsabilitatea față de animale [7].

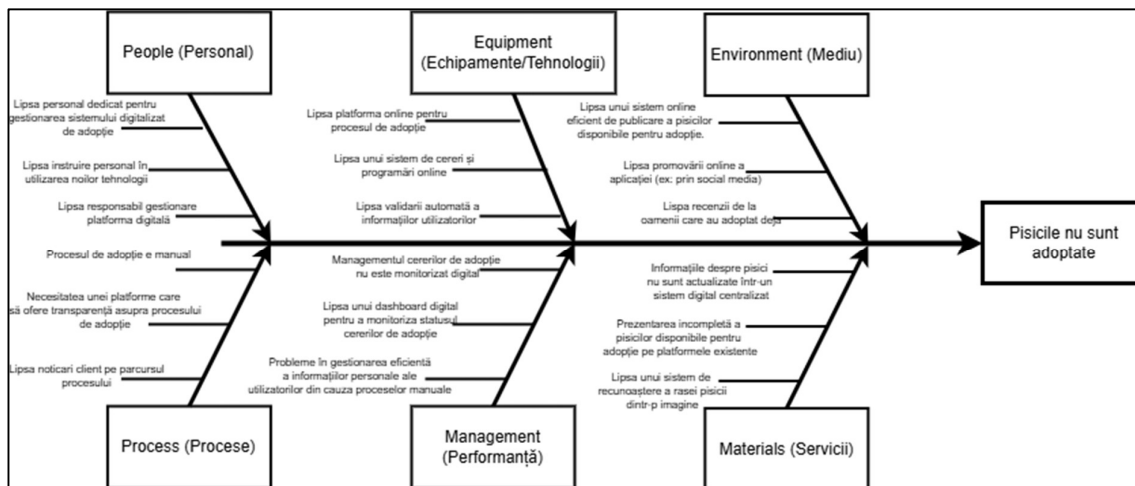
2.2 Analiza contextului

În prezent, România se confruntă cu o problemă majoră în ceea ce privește gestionarea animalelor abandonate. Motivul principal pentru care există atât de multe pisici fără un stăpân este lipsa unui sistem eficient de gestionare a acestora. În acest context, soluțiile de adopție sunt existente, dar de cele mai multe ori reprezintă procese birocratice dificile de gestionat atât pentru cei care doresc să adopte o pisică cât și pentru organizațiile care se ocupă cu salvarea acestora.

Aplicația realizată în această lucrare răspunde unei nevoi clare de digitalizare a procesului de adopție a pisicilor din România. În cadrul cercetărilor pentru acest proiect, am

investigat mai multe site-uri de adopție a pisicilor disponibile în România și am ajuns la concluzia că deși există adăposturi pentru animale și organizații de protecție a acestora, nu există prea mult platforme centralizate care să permită utilizatorilor să adopte pisici într-un mod simplu și rapid. De cele mai multe ori procesul este lung și complicat, motiv pentru care multe persoane sunt descurajate să adopte un animal.

Un exemplu relevant al acestor dificultăți poate fi regăsit în diagrama Fishbone, numită și diagrama cauză efect, din figura 1, care reprezintă principalele cauze lipsei de adopție. În urma analizării problemei, am ajuns la concluzia ca toate aceste probleme ar putea fi rezolvate prin crearea unei aplicații pentru adopția de pisici. Astfel, aplicația realizată în cadrul acestei lucrări vine ca și o soluție pentru problemele menționate.



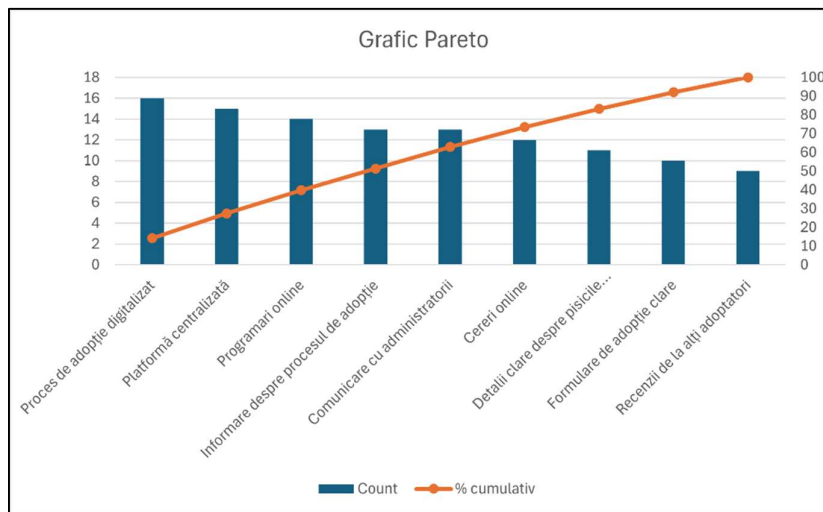
Figură 1. Diagrama Fishbone

2.3 Colectarea datelor

Pentru a-mi putea stabili ulterior obiectivele pentru aplicație a fost necesar să colectez cât mai multe date de la viitorii utilizatori, astfel încât să pot crea o aplicație care să se potrivească cu nevoile acestora. Așadar, am ales să creez un chestionar, această metodă fiind una dintre cele mai eficiente metode de colectare a datelor.

În cadrul acestui chestionar, am adresat o serie de întrebări, care pot fi regăsite în anexa 1, menite să identifice nemulțumirile și dificultățile cu care se confruntă utilizatorii în procesul actual de adopție. Scopul a fost să adun informații valoroase care să mă ajute să dezvolt o aplicație care nu doar să răspundă acestor nevoi, ci și să îmbunătățească semnificativ experiența utilizatorilor în cadrul procesului de adopție.

În urma chestionarului, am identificat principalele funcționalități și nevoi pe care le-au menționat oamenii ca și-ar dori să le regăsească într-un site de adopții de pisici și le-am folosit pentru a construi graficul Pareto, care poate fi observat și în figura 2. Pe baza rezultatelor sondajului, am selectat funcționalitățile site-ului meu pentru adopția pisicilor, prioritizând aspectele cele mai importante pentru utilizatori. Acestea includ un proces de adopție digitalizat, o platformă centralizată cu informații detaliate despre pisici, posibilitatea de a comunica ușor cu adăposturile, cereri online clare și recenzii ale altor persoane care au adoptat.



Figură 2. Grafic Pareto

2.4 Obiectivele

În continuare, mi-am stabilit obiectivele pentru această lucrare. Primul meu obiectiv a fost să creez un proces eficient prin care utilizatorii pot adopta fără dificultăți. Mi-am dorit să dezvolt o pagină dedicată pisicilor, în care acestea sunt listate împreună cu caracteristici precum genul, rasa și comportamentul fiecărei pisici. Astfel, doritorii pot vizualiza pisicile și își pot alege favorita, completând ulterior un formular de adopție pentru aceasta și programând o întâlnire.

Un alt obiectiv important a fost ca întreaga experiență să fie foarte organizată și ușor de utilizat, motiv pentru care mi-am propus să adaug și un sistem de notificări în timp real în site. Astfel, utilizatorii să fie mereu ținuti la curent cu statusul cererilor și programărilor lor, contribuind astfel la o comunicare continuă. Comunicarea dintre utilizatori și administratori am vrut să fie realizată de asemenea și prin redirectionare spre WhatsApp.

Pentru a adăuga un plus de interactivitate și transparență, am vrut să implementez și secțiunea de povești de succes, unde utilizatorii își pot încărca propriile experiențe după ce au adoptat o pisică. Aceasta nu doar că încurajează alte persoane să adopte, dar crește și încrederea

în centrul de adopție. De asemenea, foarte important pentru mine a fost și să încurajez oamenii să se implice activ în tot acest proces, de aceea am avut în plan să creez și un sistem de donații pentru a încuraja susținerea animalelor.

Totodată, mi-am dorit să realizez un proces de identificare a rasei unei pisici prin încărcarea unei fotografii. Acest proces l-am gândit special pentru a veni în sprijinul administratorilor care salvează pisici de pe stradă și trebuie să le posteze pe site.

2.5 Tipuri de utilizatori și nevoile acestora

Obiectivele descrise mai sus le-am realizat având în vedere nevoile diferitelor tipuri de utilizatori. În cadrul platformei am ajuns la concluzia ca cel mai potrivit ar fi să existe trei categorii principale de utilizatori, fiecare având acces la funcționalități diferite. Aceste interacțiuni și funcționalități au fost ilustrate printr-o diagrama Use Case, în figura 3.

Vizitatorii aplicației sunt utilizatori care nu s-au autentificat în sistem, dar care doresc să se informeze despre procesul de adopție. Nevoile lor principale sunt de a înțelege mai bine procesul de adopție pentru a putea lua decizia dacă vor să continue acest proces.

Aceștia au permisiunea de a accesa informațiile despre pisicile disponibile, dar nu au posibilitatea de a interacționa activ cu platforma pentru a completa o cerere de adopție, pentru această funcționalitate fiind necesară crearea unui cont.

Utilizatorii autentificați reprezintă persoanele care au conturi pe platformă și care pot interacționa activ cu aplicația. Nevoile acestora sunt de a avea acces personalizat la aplicație. În plus față de permisiunile pe care le au vizitatorii, acest tip de utilizatori au posibilitatea de a adopta propriu-zis pisicile dorite. Pot să își salveze pisicile favorite, să completeze formularul de adopție și în urma aprobării cererii, pot să își programeze o vizită la centrul de adopție. De asemenea, acești utilizatori mai au și posibilitatea de a adăuga o poveste cu experiența lor personală și de a efectua donații. Pentru a asigura o comunicare continuă cu administratorii platformei, acești utilizatori primesc notificări legate de starea cererilor și programărilor lor și au posibilitatea de a fi redirecționați pe WhatsApp pentru a discuta cu un administrator direct.

Administratorii sunt cei care au acces complet la toată aplicația. Aceștia au nevoie de un control complet asupra platformei, pentru a gestiona eficient conținutul aplicației. Aceștia sunt responsabili pentru gestionarea tuturor pisicilor disponibile pentru adopție, aprobarea cererilor de adopție, programărilor și a poveștilor de succes ale utilizatorilor. De asemenea, administratorii au acces la statistici legate de fiecare componentă a aplicației. Pentru a simplifica procesul de adăugare a detaliilor pentru fiecare pisică, administratorii au acces și la o funcționalitate prin care pot determina rasa unei pisici pe baza unei poze încărcate de ei.

APLICAȚIE WEB ÎN LARAVEL PENTRU ADOPTȚIA PISICILOR

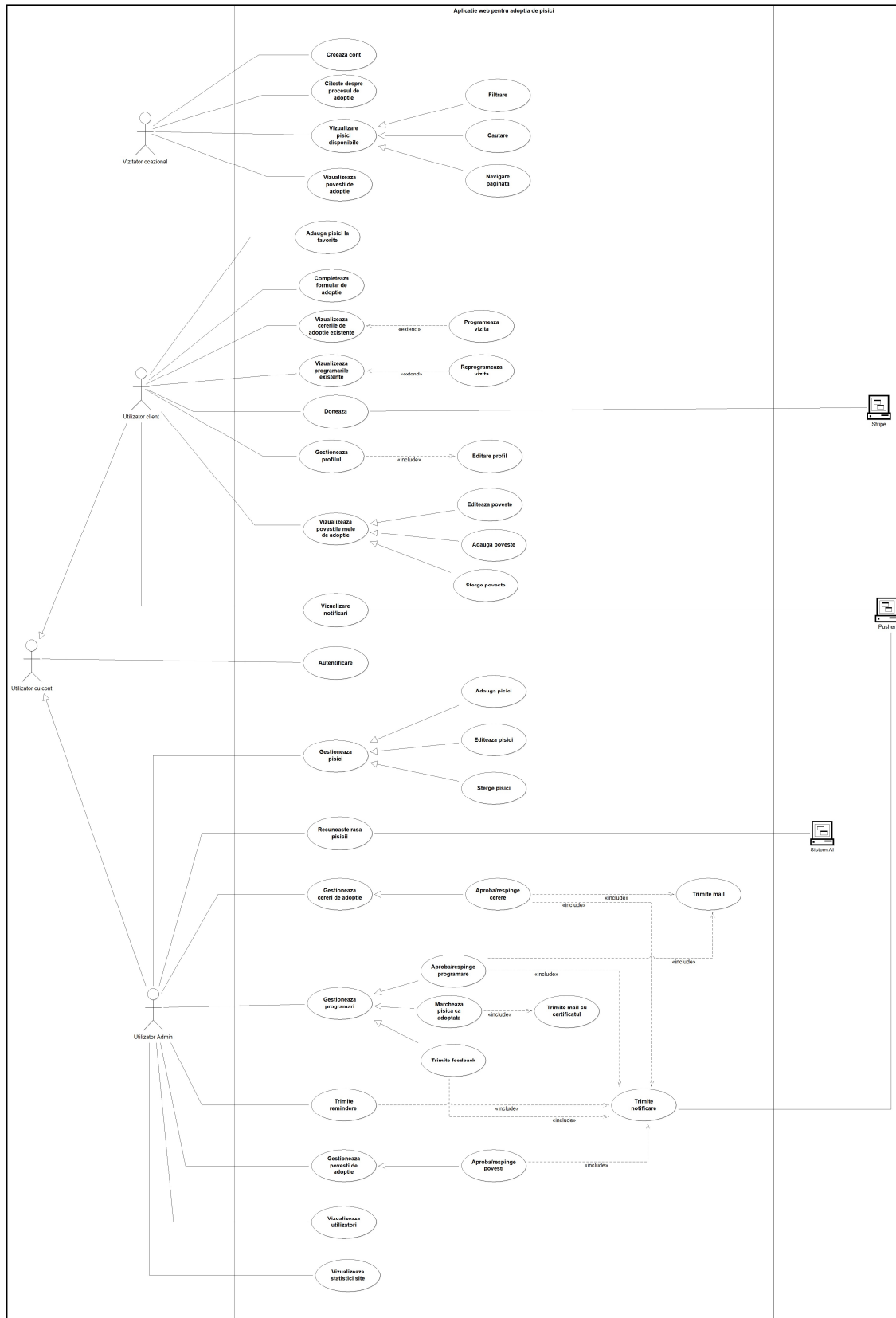


Figura 3. Diagrama Use Case a aplicației

2.6 Analiza proceselor

Procesul de adopție al unei pisici reprezintă elementul central al aplicației mele web, fiind punctul de legătură al tuturor celorlalte funcționalități. Prin urmare, pentru o reprezentare clară al acestui proces, am realizat doua diagrame BPMN (Business Process Model and Notation), fiind un standard folosit pentru modelarea proceselor de business.

2.6.1 Procesul As-Is

Diagrama BPMN As-Is prezentată în figura 4, reflectă procesul tradițional, predominant manual, al unei adopții de pisici. Am realizat această diagramă pentru a surprinde cum decurge o adopție în momentul de față înainte de implementarea unui proces digitalizat

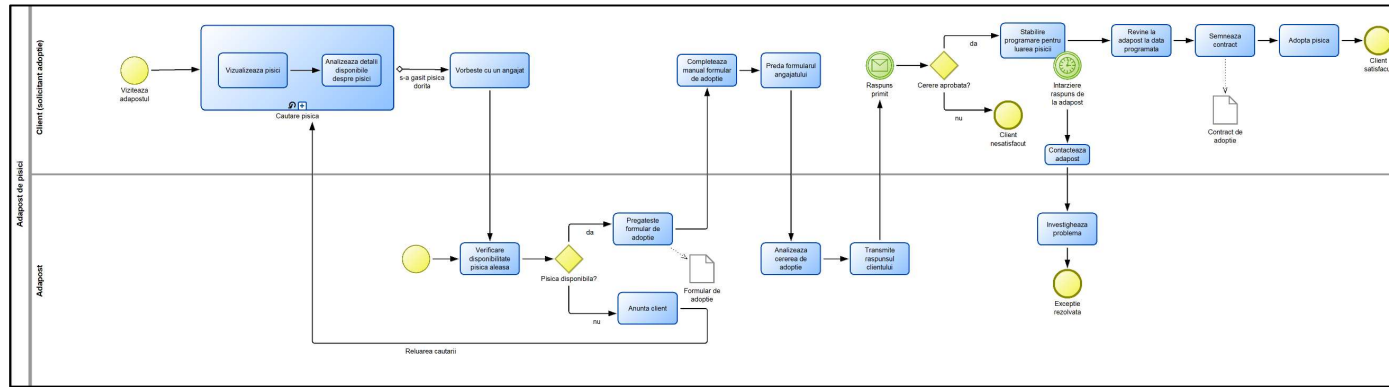
Fiind un proces bazat strict pe interacțiuni directe dintre oameni, pot apărea diferite probleme, cum ar fi întârzieri în comunicarea răspunsurilor din partea angajaților sau lipsa disponibilității pisicii dorite de client. Aceste aspecte evidențiază clar nevoia unei platforme digitale care să simplifice și să eficientizeze întregul proces.

2.6.2 Procesul To-Be

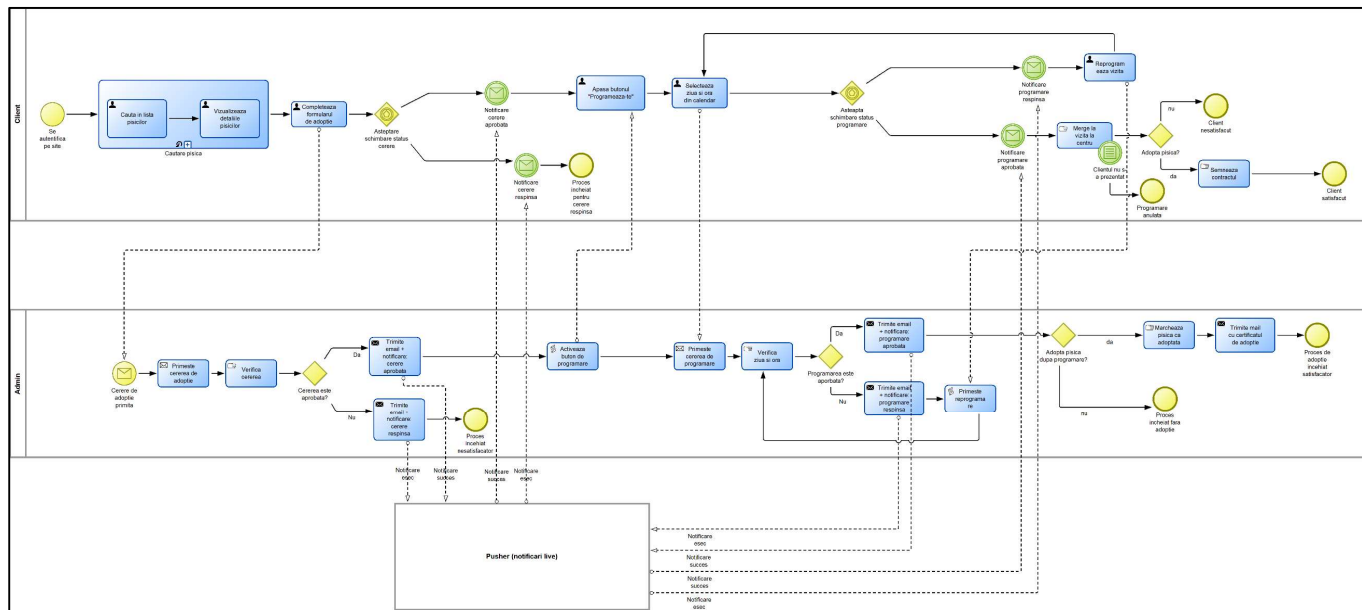
Diagrama BPMN To-Be, din figura 5, reprezintă o versiunea digitalizată și automatizată a aceluiași proces, exact așa cum mi-am propus să implementez în cadrul aplicației web din această lucrare. În această formă, dispar problemele legate de întârzieri sau lipsa de comunicare, deoarece totul se desfășoară prin intermediul aplicației, iar utilizatorii sunt informați în timp real despre statusul cererilor sau programărilor, prin notificări automate. De asemenea, clientul care își dorește să adopte o pisică nu mai este nevoit să se prezinte la adăpost pentru fiecare pas al adopției deoarece are posibilitatea de a completa formularul de adopție și de a programa o întâlnire online.

Am realizat această diagramă pentru a evidenția cât de mult optimizează o aplicație de adopție de pisici atât experiența pentru utilizator cât și pentru administrație. Procesul devine mai clar, mai rapid și mai ușor de gestionat, eliminând birocrăția, reducând erorile umane și oferind o transparență mult mai mare.

APLICAȚIE WEB ÎN LARAVEL PENTRU ADOPTIA PISICILOR



Figură 4. Diagrama BPMN As-Is



Figură 5. Diagrama BPMN To-Be

3. Proiectarea sistemului informatic

Capitolul următor descrie structura internă a sistemului informatic. Vor fi prezentate modelele de date, arhitectura aplicației și comportamentul său prin diferite diagrame UML, toate acestea contribuind la înțelegerea modului în care funcționează aplicația în ansamblu.

3.1 Proiectarea bazei de date

Orice aplicație informatică are nevoie de un set de elemente interconectate pentru colectarea, transmiterea, stocarea și procesarea datelor. Acest rol este îndeplinit de un Sistem de Baze de Date (SBD) care cuprinde atât datele structurate cât și programele pe care le gestionează. Datele sunt structurate și stocate pe calculator, în memoria externă cu un Sistem de Gestiune a Bazelor de Date (SGBD), care reprezintă un software specializat [8].

MySQL este cel mai popular sistem de gestiune a bazelor de date (SGBD) open-source, recunoscut datorită fiabilității, performanței și ușurinței sale în utilizare. Deoarece este o bază de date relațională, MySQL organizează informațiile în tabelele formate din rânduri și coloane structurate în scheme care definesc modul de organizare și stocare a datelor. Aceste caracteristici permit modificările datelor să fie efectuate într-un mod consistent și fiabil, chiar și în cazul unei defecțiuni și gestionarea unor baze de date foarte mari [9]. Acestea sunt motivele principale pentru care am ales să proiectez baza de date a aplicației mele folosind MySQL.

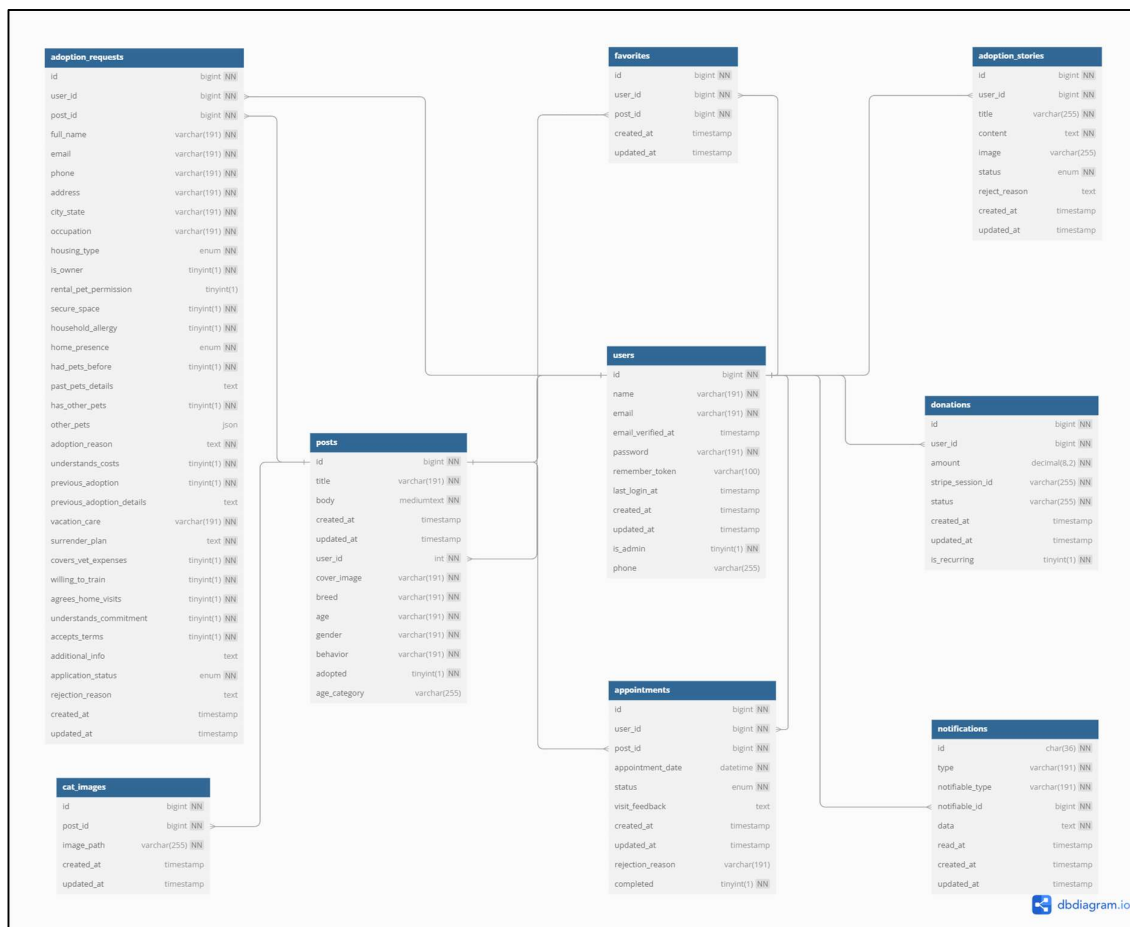
După cum se poate observa în figura 6, am realizat diagrama entitate-relație (ERD), pentru a reprezenta vizual tabelele din baza mea de date. Această diagramă reflectă structura logică a datelor, punând accent pe relațiile dintre entitățile cheie ale sistemului.

Entitatea centrală a aplicației este “posts”, care conține informațiile despre pisicile disponibile pentru adopție (nume, rasă, vârstă, comportament, imagini, galerie de imagini etc.). Pentru a permite atașarea unei galerii de imagini fiecărei pisici, am adăugat și o tabelă secundară “cat_images”, care este într-o relație de tip one-to-many cu tabela “posts”.

Relațiile dintre tabela “users”, care reprezintă utilizatorii autentificați în sistem, și celelalte tabele reflectă acțiunile pe care aceștia le pot realiza în aplicație. Mai exact, fiecare utilizator poate trimite o cerere de adopție („adoption_requests”), poate salva pisici în lista de favorite („favorites”), poate publica o poveste de adopție („adoption_stories”) și își poate face

o programare pentru vizită („appointments”). Aceste relații sunt de tip one-to-many, fiecare utilizator putând avea mai multe cereri, favorite, povești sau programări.

Tabelele “donations” și “notifications” care reprezintă donațiile și notificările sunt și ele legate tot de utilizatori, prin relații one-to-many deoarece doar utilizatorii autentificați pot realiza donații sau primi notificări. Aceste relații sunt realizate prin intermediul cheii externe user_id, care permite identificarea utilizatorului asociat fiecărei acțiuni.



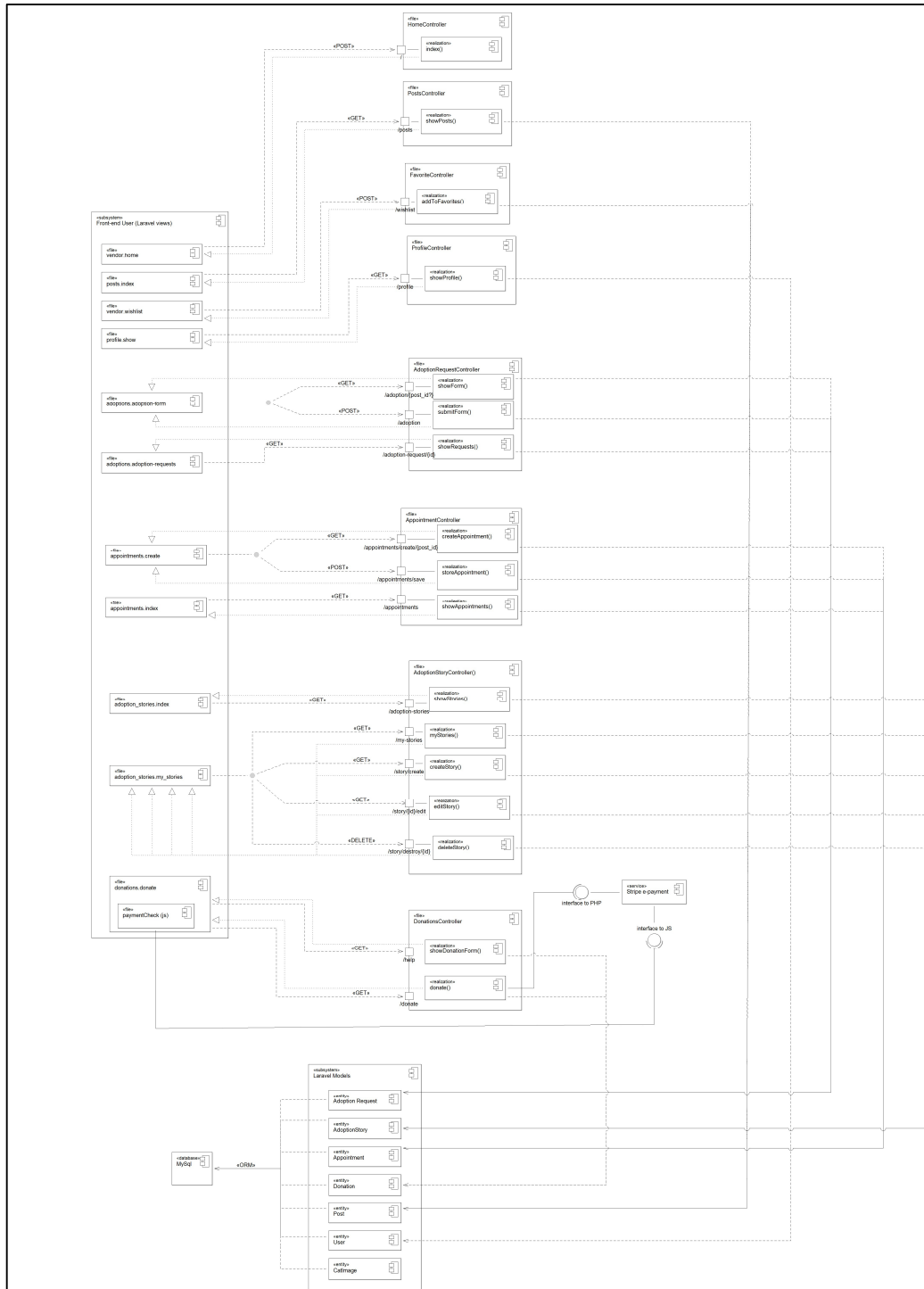
Figură 6. Diagrama entitate-relație (ERD)

3.2 Arhitectura generală a sistemului

La baza arhitecturii generale a sistemului stau controllerele, funcții definite în cadrul acestora și legătura lor directă cu modelele și paginile vizibile în aplicație. Fiecare componentă este realizată conform structurii specifice arhitecturii MVC (Model-View-Controller) oferite de Laravel. Pentru a reprezenta această structură am realizat diagrama de componente.

Pe partea de utilizator, așa cum se poate observa și în figura 7, componentele sunt structurate în jurul controllerelelor care gestionează toate funcționalitățile disponibile pentru

utilizatori. Fiecare controller cuprinde metode specifice care sunt accesate prin rute HTTP de tip GET sau POST. Aceste metode sunt apelate de componentele din interfața aplicației (Blade views), stabilind o legătură clară între acțiunile utilizatorului și logica aplicației. De asemenea, se poate observa și integrarea cu servicii externe precum Stripe.



Figură 7. Diagrama de componente pe partea de utilizator

Pe partea de administrator, după cum se poate observa în figura 8, componentele sunt organizate în jurul controllerelor din namespace-ul Admin. La fel ca și la utilizatori, am reprezentat toate controllerele legate de funcționalitățile adminului, împreună cu metodele dedicate și rutele HTTP corespunzătoare. De asemenea, am integrat în diagramă și interacțiunea aplicației cu serviciile externe folosite pentru funcționalitățile adminului, mai exact interacțiunea interfeței cu sistemul de predicție a rasei bazat pe inteligență artificială.



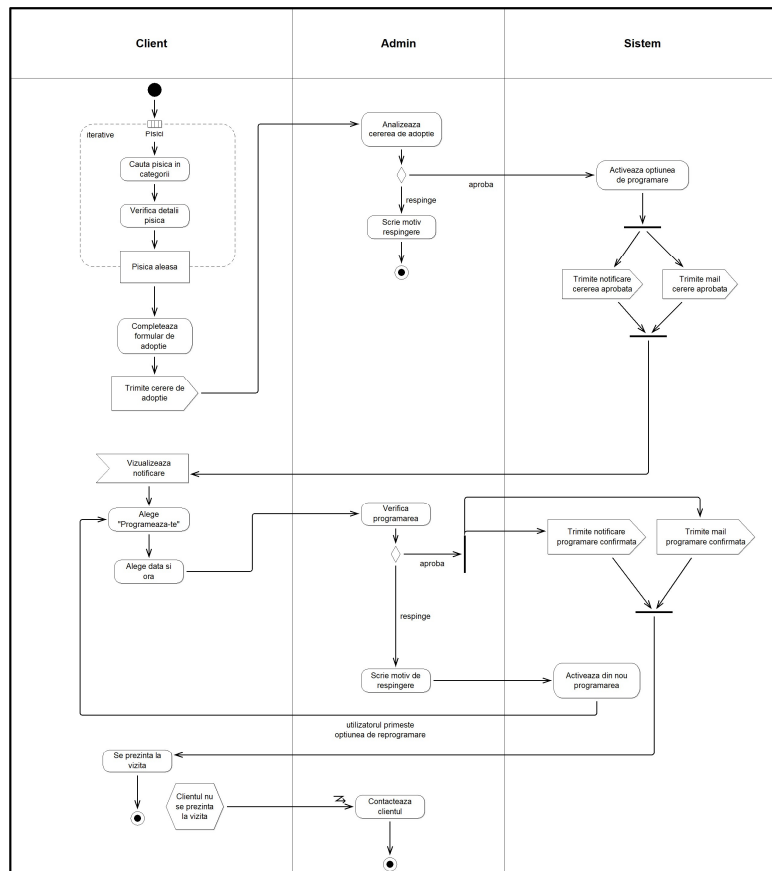
Figură 8. Diagrama de componente pe partea de admin

3.3 Modelarea comportamentului aplicației

Modelarea comportamentului aplicației are rolul de evidenția modul în care aceasta reacționează la acțiunile utilizatorilor. Prin descrierea modului în care sunt gestionate stările, activitățile și interacțiunile aplicației, se poate înțelege mai ușor cum se desfășoară lucrurile în aplicație.

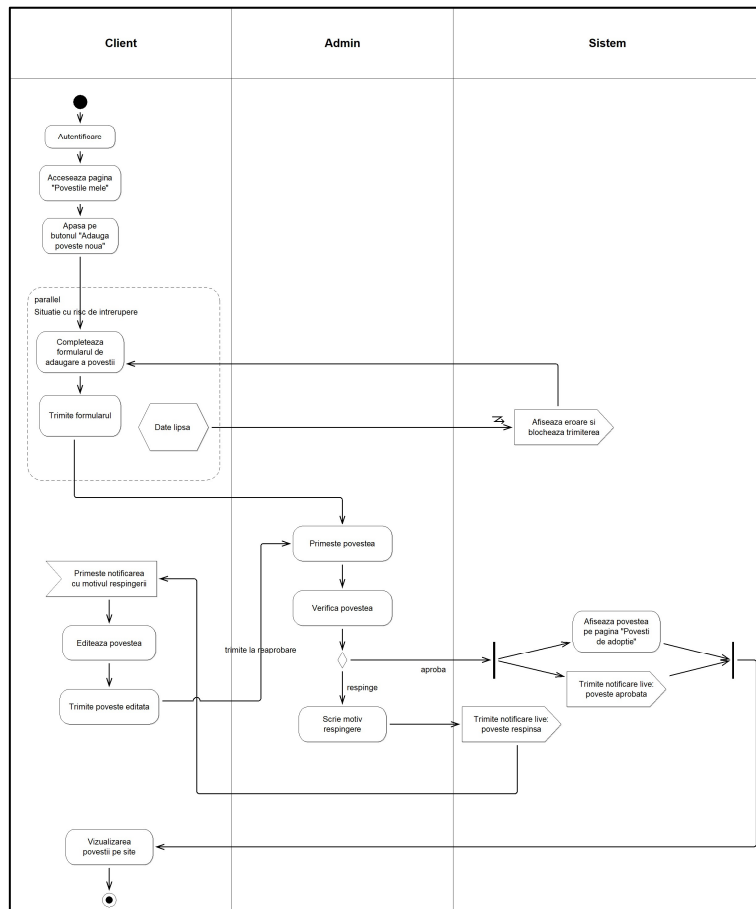
Procesul trimerii unei cereri de adopție și realizarea unei programări este esențial pentru aplicația mea. Astfel, în diagrama de activități, din figura 9, se poate observa acest proces și toate etapele acestuia, de la căutarea pisicii dorite până la adoptarea acesteia. Am folosit simboluri specifice acestei modelări precum acțiuni, decizii, ramificații și interacțiuni între actorii implicați (client, admin și sistem) pentru a reprezenta procesele diferite.

Totul pornește de la utilizator, care caută o pisică, completează formularul de adopție și trimite cererea. Aceasta ajunge la admin, care o analizează și decide dacă o aprobă sau o respinge. Diagrama arată apoi cum reacționează sistemul, trimite notificări, activează calendarul de programare sau permite reprogramarea, în funcție de deciziile luate. Astfel, se reflectă clar traseul fiecărei acțiuni și cum aplicația răspunde automat sau manual.



Figură 9. Diagrama de activități pentru procesul de adopție

Un alt proces complex, foarte important pentru aplicația mea, este cel de adăugare a unei povești de adopție. Pentru a reprezenta vizual și acest proces, am realizat diagrama de activități care poate fi regăsită în figura 10. Totul pornește cu autentificarea și accesarea secțiunii “Poveștile mele”, de unde clientul poate completa și trimite formularul poveștii spre aprobare. În diagramă am inclus și posibilele erori, cum ar fi situațiile în care lipsesc date și sistemul oprește astfel trimiterea. În funcție de verificarea făcută de administrator, povestea este poate fi aprobată și afișată pe site, sau poate să fie respinsă, caz în care utilizatorul primește un motiv și poate reface povestea.

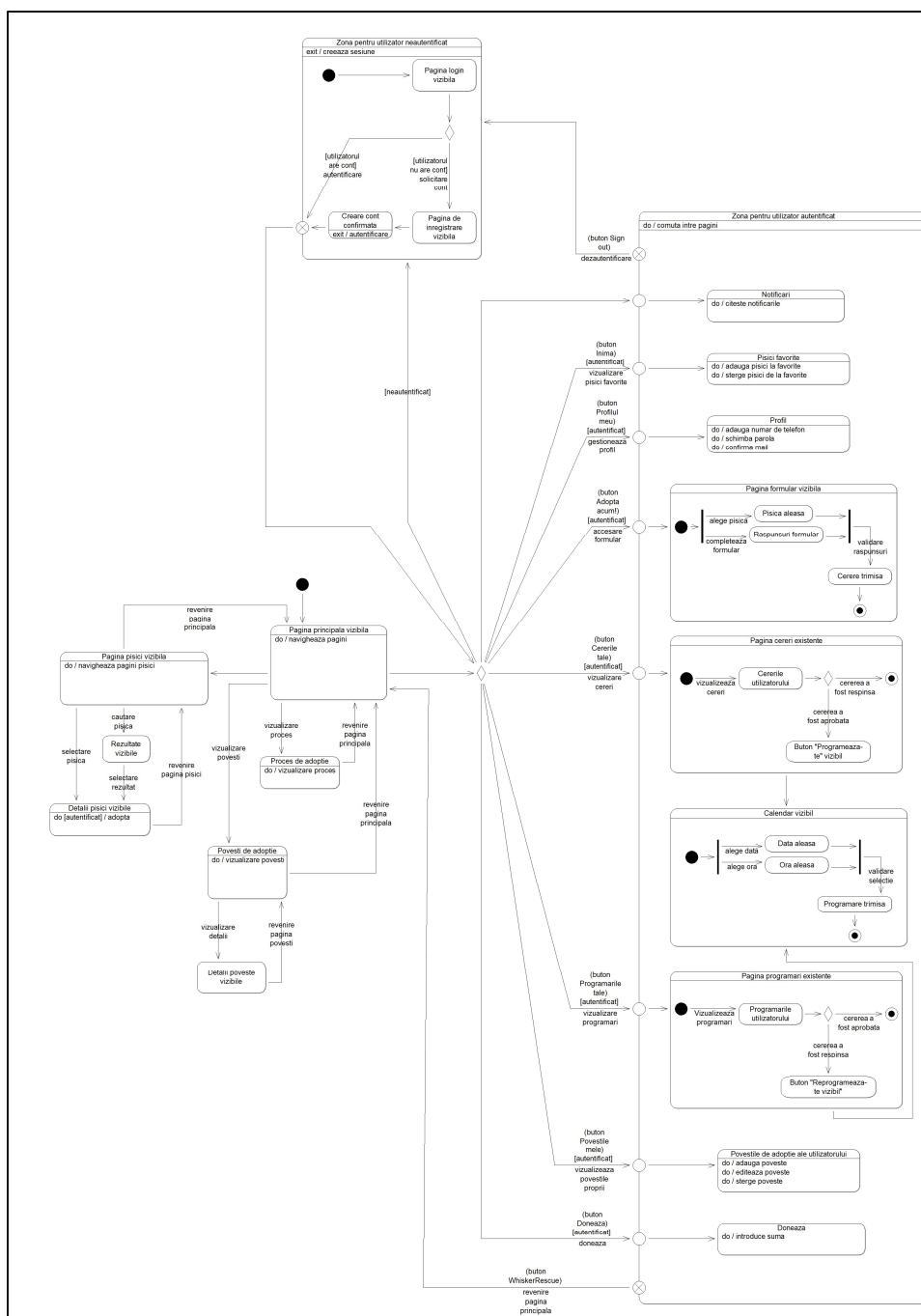


Figură 10. Diagrama de activități pentru procesul de povești de adopție

Pentru a evidenția modul în care aplicația reacționează în funcție de acțiunile utilizatorului și condițiile fiecărei stări din aceasta, am realizat diagrama de stări, care poate fi vizualizată în figura 11. Fiecare stare din această diagramă reprezintă o anumită pagină vizibilă din aplicație, cum ar fi pagina principală, pagina de login sau pagina de completare a

formularului, iar tranzițiile dintre ele sunt determinate de utilizatori prin acțiuni precum apăsarea unui buton.

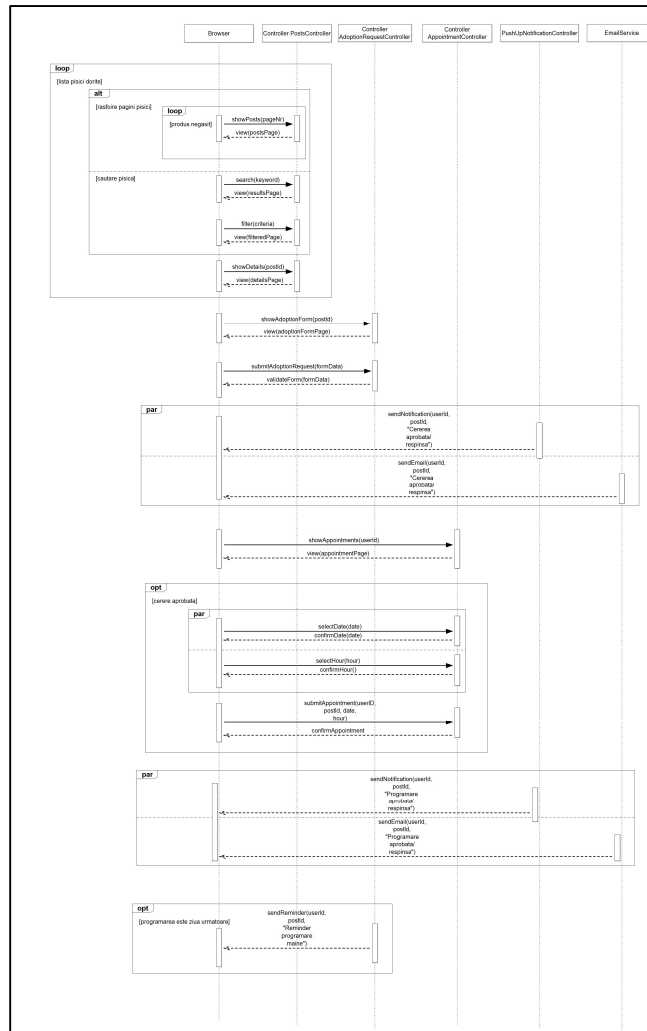
Am realizat în diagramă o diferență clară între diferențele de comportament ale aplicației pentru utilizatorii autentificați și cei neautentificați. De exemplu, un utilizator care nu este logat poate accesa doar anumite pagini, în timp ce după autentificare are acces la funcționalități avansate precum cereri, programări, povești și donații.



Figură 11. Diagrama de stări

Pentru a înțelege funcționarea unei aplicații, este necesar să analizăm cum sunt transmise mesajele și răspunsurile între diferitele componente implicate, iar acest lucru poate fi vizualizat într-o diagramă de secvență. În figura 12, se poate observa diagrama de secvență a aplicației mele în care este descrisă comunicarea dintre browser și controllerele care se ocupă de logica din spatele interfeței utilizatorului.

Procesul descris în această diagramă este cel de completare a unei cereri de adopție și programare ulterioară. Procesul începe de la acțiunile utilizatorului în interfață care declanșează o serie de mesaje trimise către controllerele din back-end. Odată ce cererea este validată și procesată, sistemul generează automat notificări live și emailuri, pentru a informa utilizatorul cu privire la statusul cererii. Același mecanism se aplică și în etapa de programare, unde confirmările sau reminderurile sunt transmise prin email și notificări, asigurând o comunicare eficientă și în timp real între aplicație și utilizator.



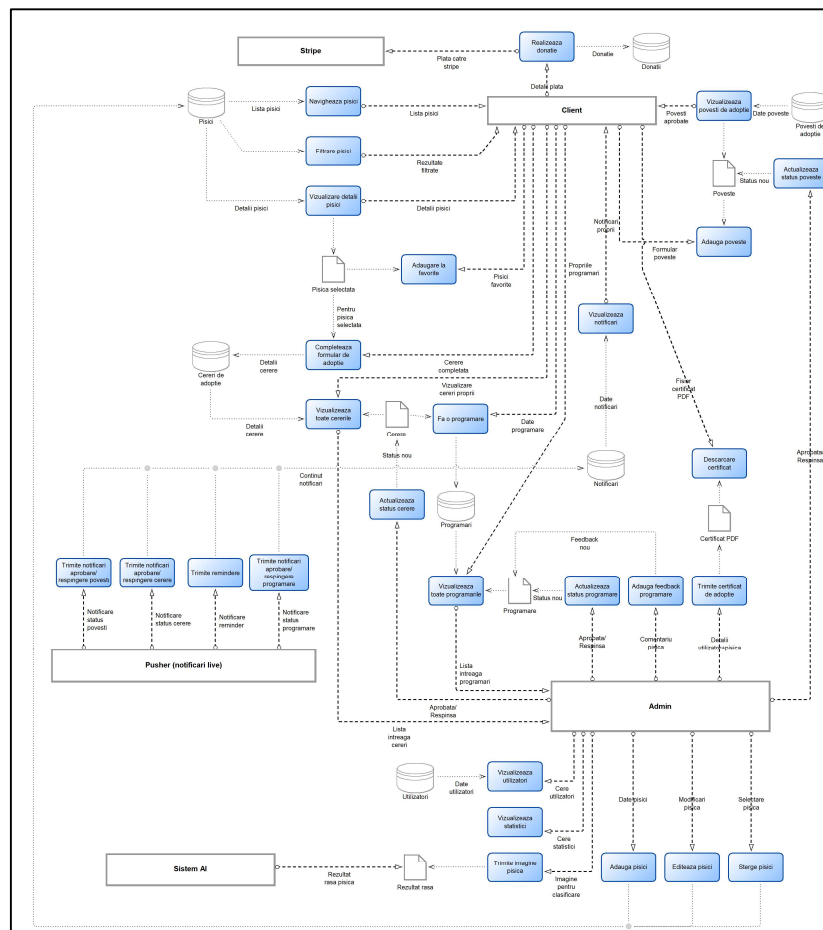
Figură 12. Diagrama de secvență

3.4 Proiectarea fluxurilor de date

Pentru o înțelegere mai bună a modului în care circulă datele în cadrul aplicației mele, am realizat diagrama fluxurilor de date (DFD), care oferă o viziune de ansamblu asupra entităților principale, proceselor și interacțiunilor dintre utilizatori, sistem și baze de date.

Așa cum se poate observa în figura 13, în partea superioară a diagramei, sunt reprezentate toate acțiunile pe care un utilizator le poate face în aplicație. Toate acțiunile realizate de acesta, duc la generarea și transmiterea de date către baza de date, iar unele informații sunt preluate și de administrator pentru aprobare sau procesare. În diagrama am ilustrat și două servicii externe importante, Stripe, responsabil de procesarea donațiilor, și Pusher, care asigură trimiterea notificărilor live către utilizatori.

În partea inferioară a diagramei, am reprezentat acțiunile și responsabilitățile administratorului, care gestionează datele provenite de la utilizatori. Toate aceste acțiuni implică interacțiuni directe cu baza de date și declanșează, în unele cazuri, notificări live prin Pusher, oferind un flux coordonat de date între partea de administrare și aplicație.



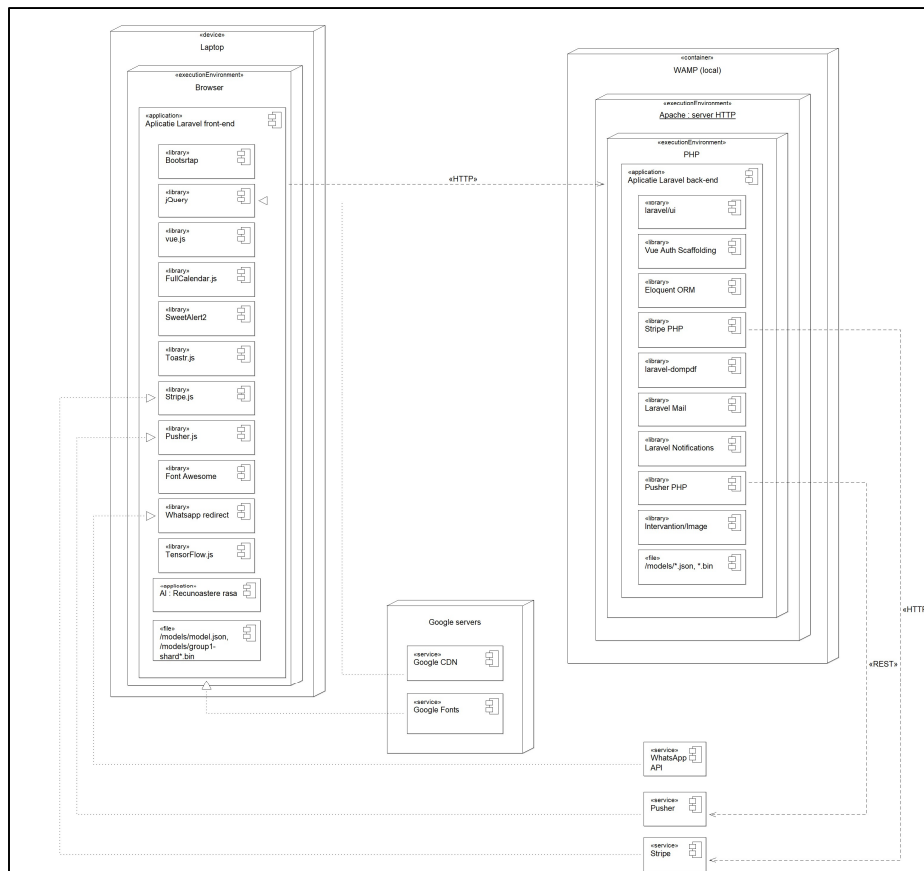
Figură 13. Diagrama fluxurilor de date (DFD)

3.5 Infrastructura de implementare

Pentru a descrie modul în care componentele aplicației sunt distribuite am realizat diagrama de deployment, care poate fi vizualizată în figura 14. La baza arhitecturii se află două componente principale, mai exact partea de front-end, care rulează în browserul utilizatorului pe un laptop și partea de back-end, care rulează pe un server local WAMP cu Apache și PHP.

Front-end-ul aplicației este construit în Laravel Blade și se bazează pe o serie de biblioteci care se pot observa în figura 14. În plus, am integrat și serviciile externe din aplicație precum Stripe.js, Pusher.js și redirecționarea către WhatsApp printr-o bibliotecă dedicată. Pe lângă acestea, aplicația utilizează resurse externe găzduite pe serverele Google, cum ar fi Google Fonts și Google CDN, pentru o încărcare rapidă și un aspect vizual modern.

Back-endul este și el compus din mai multe biblioteci pe care le-am folosit pentru lucruri precum autentificarea (Vue Auth Scaffolding), trimiterea de emailuri (Laravel Mail), generarea de PDF-uri (laravel-dompdf) și interacțiunea cu baza de date prin Eloquent ORM. Acesta comunică cu servicii externe precum Stripe, Pusher și WhatsApp API, folosind protocoalele HTTP și REST.



Figură 14. Diagrama de deployment

4. Implementarea

În acest capitol, voi detalia modul în care am transpus în practică ideile și cerințele analizate anterior, construind funcționalitățile aplicației pas cu pas. Am structurat codul astfel încât să fie clar, eficient și ușor de extins, folosind Laravel pentru partea de back-end și tehnologii moderne de front-end.

4.1 Componente funcționale ale aplicației

4.1.1 Back-End

Partea de Back-End a unei aplicații reprezintă logica din spate, interacțiunea cu baza de date și manipularea datelor. Back-End-ul nu interacționează direct cu utilizatorul, acesta asigură corectitudinea operațiunilor și persistența datelor din spate. Astfel, acesta gestionează cererile primite de la interfața utilizatorului (Front-End), le procesează conform regulilor definite și returnează răspunsuri adecvate. În continuare, voi prezenta structura componentelor de Back-End utilizate în cadrul aplicației mele.

Serverul Web

Partea de server, sau Back-End-ul, rulează pe servere web care au rolul de a primi cererile de la clienți, de a le procesa și returna răspunsurile corespunzătoare. Această componentă a aplicației gestionează autentificarea, manipularea datelor, aplicarea logicii de business și comunicarea cu bazele de date [10]. Apache HTTP Server este unul dintre cele mai utilizate servere de web și e utilizat în mod obișnuit în stivele LAMP (Linux, Apache, MySQL, PHP) fiind compatibil cu diverse sisteme de operare, ceea ce îl face o alegere potrivită pentru dezvoltatorii web [11]. Fiind compatibil cu framework-ul Laravel, am ales Apache pentru rularea aplicației în mediul de producție.

Protocoalele HTTP

Pe web, comunicarea este determinată de o varietate de protocoale, fiecare având roluri specifice. Cel mai important dintre acestea este Hyper-Text Transfer Protocol (HTTP) și reprezintă standardul de comunicare utilizat în aplicațiile web pentru transmiterea cererilor și

răspunsurilor între client (browser) și server. Comunicarea în HTTP urmează modelul cerere-răspuns, în care clientul trimite serverului o cerere pentru a realiza o acțiune, iar serverul, la rândul său, răspunde cu rezultatul solicitării [12].

REST – Transferul Stării de Reprezentare

Transferul Stării de Reprezentare este un stil de arhitectură care definește un set de principii pentru dezvoltarea de servicii web. Acesta se bazează pe modul în care protocolul HTTP a fost conceput să fie utilizat, încurajând o abordare clară a interacțiunilor client-server. Într-un sistem RESTful, informațiile sunt împărțite în resurse, unde fiecare resursă este identificată printr-un URI (Identificator de Resurse Uniform) unic. Operațiunile asupra acestor resurse se realizează prin metodele HTTP standard (GET, POST, PUT, DELETE). Această arhitectură este frecvent utilizată în dezvoltarea de aplicații dezvoltate cu framework-uri precum Laravel [13].

Laravel

Laravel este cel mai utilizat framework PHP atât pentru programatori începători cât și pentru cei avansați. Framework-urile PHP ajută la promovarea “Dezvoltării de Aplicații Rapide” (RAD) economisind timp, contribuind la crearea de aplicații mai stabile și reducând cantitatea de codare repetitivă pentru dezvoltatori [14].

Laravel se bazează pe arhitectura MVC. Model-View-Controller (MVC) este un model de design care conține trei componente principale. Modelul (Model) este responsabil pentru manipularea datelor și interacțiunea cu baza de date. Vederea (View) prin intermediul fișierelor Blade (un motor care permite scrierea de cod HTML) se ocupă de logica folosită pentru prezentarea datelor în interfața utilizatorului. Controlerul (Controller) funcționează ca un canal de comunicare între model și vedere. Acesta primește datele de intrare, le procesează, apelează logica necesară prin modele și returnează un răspuns adecvat printr-o vedere care conține datele pregătite pentru afișare [15].

4.1.2 Front-End

Partea de Front-End a unei aplicații reprezintă interfața vizibilă cu care utilizatorul interacționează direct. Aceasta include toate elementele grafice dintr-o aplicație, având rolul de a asigura o experiență adecvată. Front-End-ul transmite cereri inițiate de către utilizatori spre Back-End și afișează datele primite într-o manieră interactivă. În continuare, voi prezenta structura componentelor de Front-End utilizate în cadrul aplicației dezvoltate în această lucrare.

Comunicarea cu Back-End-ul

Conectarea între Front-End și Back-End se realizează printr-o serie de protocoale și mecanisme de comunicare. Principalul mijloc de interacțiune este prin intermediul cererilor și răspunsurilor HTTP.

Atunci când utilizatorul efectuează o acțiune într-o aplicație web, browserul trimite o cerere HTTP către server. Serverul prelucrează această cerere, accesează datele necesare din baza de date, dacă este cazul și ulterior trimite un răspuns HTTP înapoi [16].

HTML

HTML este prescurtarea de la Hypertext Markup Language (Limba de Marcare Hipertext) și reprezintă limbajul de bază utilizat în World Wide Web (WWW). Acesta nu este un limbaj de programare propriu-zis, ci un limbaj de marcare folosit pentru a permite dezvoltatorilor să definească componente multimedia [2].

JavaScript

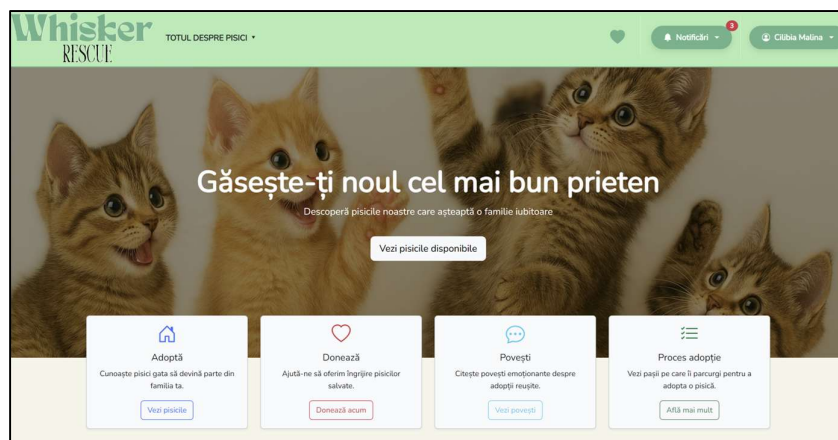
JavaScript este un limbaj de programare orientat pe prototipuri, utilizat în principal pentru adăugarea de interactivitate și funcționalități dinamice pe paginile web. Limbajul a fost adoptat de toate browserele grafice majore, permițând dezvoltarea aplicațiilor web moderne, care permit interacțiunea directă fără a fi necesară reîncărcarea paginii pentru fiecare acțiune. Javascript poate fi executat direct în browser și interacționează cu HTML și CSS pentru a modifica conținutul și stilurile paginii, răspunzând la acțiunile utilizatorului în timp real [4].

CSS

CSS (Cascading Style Sheets – Foi de stil în cascadă) este un limbaj de stilizare utilizat pentru definirea aspectului vizual al unui document HTML sau XML. Acesta permite dezvoltatorilor să controleze stilul paginilor web, specificând modul în care elementele trebuie afișate în browser. CSS este suportat de toate browserele moderne, fiind astfel considerat o tehnologie fundamentală a dezvoltării web [2].

4.2 Interfața utilizatorilor

Imaginea de mai jos reprezintă pagina principală afișată automat atunci când un utilizator accesează aplicația web. Interfața este construită cu HTML, CSS și Bootstrap pentru a crea o experiență vizuală plăcută. Meniul superior permite accesul rapid la notificări, contul utilizatorului și lista de favorite. Secțiunea centrală include un mesaj de întâmpinare și un buton de redirectionare spre lista pisicilor disponibile spre adopție. În partea inferioară, sunt afișate patru carduri interactive, fiecare fiind legat de funcționalități esențiale ale aplicației.



4.2.1 Cerere de adopție

Formularul de adopție este unul dintre cele mai importante puncte de interacțiune între utilizator și aplicație, acesta colectând informații despre persoana care dorește să adopte o pisică. Pentru ca procesul să se desfășoare corect, fiecare câmp din formular este constrâns de niște reguli de validare. Atunci când datele introduse nu respectă cerințele impuse utilizatorul este informat imediat prin mesaje de eroare, așa cum se poate observa în exemplul de mai jos.

În urma completării formularului de adopție, utilizatorii sunt redirecționați pe o pagină unde aceștia pot să își vizualizeze toate cererile de adopție. Această pagină se poate vedea în imaginea de mai jos. Utilizatorii pot vizualiza detaliile fiecărei cereri, statusul acestora și motivul de respingere, doar dacă este cazul.



Pentru ca acest formular să funcționeze corect, în spate se află o logică solidă de validare și procesare a datelor, implementată în controllerul `AdoptionRequestController` al aplicației. Metoda `submitForm()` este cea care gestionează procesul prin care un utilizator trimite o cerere de adopție. La trimiterea formularului, datele sunt validate automat folosind funcția `validate()`, care verifică dacă toate câmpurile esențiale sunt completate corect. Se controlează formatul emailului, existența postării selectate, formatul numărului de telefon, completarea motivului pentru adopție și acceptarea termenilor. Unele câmpuri pot fi opționale, cum ar fi detaliile despre alte animale sau adopții anterioare, dar cele mai importante condiții sunt impuse prin reguli stricte de validare.

După ce toate datele trec de validare, lista cu alte animale este convertită în format JSON, iar cererea este legată de utilizatorul autentificat prin adăugarea `user_id`. Cererea este apoi salvată în baza de date, iar utilizatorul primește un mesaj de confirmare. Acest proces contribuie la colectarea unor informații complete și corecte, esențiale pentru luarea unei decizii de administratori privind adopția.

```
public function submitForm(Request $request)
{
    $validatedData = $request->validate([
        'post_id' => 'required|exists:posts,id',
        'full_name' => 'required|string|max:255',
        'email' => 'required|email|max:255',
```

```

'phone' => 'required|string|regex:/^[0-9+\s()-]{7,20}$/',
'address' => 'required|string|max:255',
'city_state' => 'required|string|max:255',
'occupation' => 'required|string|max:100',
'housing_type' => 'required|string',
'is_owner' => 'required|boolean',
'rental_pet_permission' => 'nullable|boolean',
'secure_space' => 'required|boolean',
'had_pets_before' => 'required|boolean',
'past_pets_details' => 'nullable|string',
'adoption_reason' => 'required|string|min:10',
'understands_costs' => 'required|boolean',
'previous_adoption' => 'required|boolean',
'previous_adoption_details' => 'nullable|string',
'vacation_care' => 'required|string|min:5',
'has_other_pets' => 'required|boolean',
'other_pets' => 'nullable|array',
'household_allergy' => 'required|boolean',
'home_presence' => 'required|string',
'surrender_plan' => 'required|string|min:10',
'covers_vet_expenses' => 'required|boolean',
'willing_to_train' => 'required|boolean',
'agrees_home_visits' => 'required|boolean',
'understands_commitment' => 'required|boolean',
'accepts_terms' => 'required|boolean',
'additional_info' => 'nullable|string',
]);

$validatedData['other_pets'] = json_encode($request->other_pets ?? []);
$validatedData['user_id'] = auth()->id();

AdoptionRequest::create($validatedData);

return redirect()->back()->with('message', 'Cererea de adopție a fost trimisă cu
succes!');
}

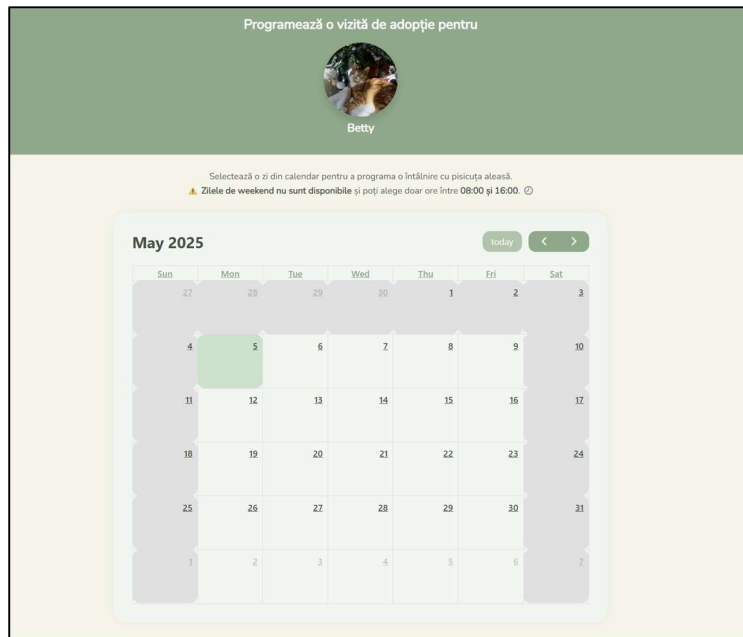
```

4.2.2 Programări

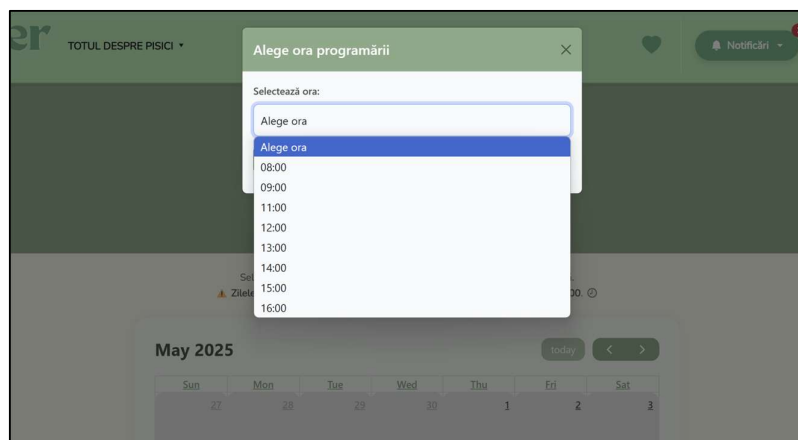
În cadrul aplicației, funcționalitatea de programare a unei vizite de adopție joacă un rol esențial în tot procesul de adopție. Pentru a realiza acest pas și pentru a evita conflictele de programări, am implementat un calendar interactiv în care utilizatorii pot selecta ziua și ora dorită pentru vizită.

În imaginea de mai jos se poate observa cum arată calendarul interactiv pentru utilizatori. Ziua curentă este evidențiată cu culoarea verde, pentru a facilita identificarea rapidă a datei curente, iar zilele care nu sunt disponibile pentru programare (zilele din trecut și

weekend-urile) sunt colorate cu gri. Acest sistem de colorare ajută utilizatorii să vizualizeze rapid zilele disponibile și să evite selectarea unor zile care nu sunt premise.



După cum poate fi observat și în imaginea de mai jos, după ce utilizatorul selectează data pentru programare, un modal va apărea pe ecran, oferindu-i posibilitatea de a alege ora dorită pentru vizită. În acest modal, sunt afișate doar orele disponibile, excluzându-le pe cele deja rezervate pentru acea zi. De asemenea, dacă nu sunt ore disponibile, utilizatorul va fi informat printr-un mesaj clar, iar câmpul pentru selecția orei va fi dezactivat.



Calendarul interactiv care apare în interfața utilizatorilor l-am realizat utilizând biblioteca FullCalendar. Codul de mai jos se ocupă doar cu încărcarea și inițializarea calendarului, astfel acesta este încărcat printr-o instanță a clasei FullCalendar.Calendar,

asociată cu un element HTML specificat, iar evenimentele sunt încărcate dintr-o sursă externă folosind o rută definită în Laravel (appointments.fetch). Configurările referitoare la ora și dată, sunt realizate cu ajutorul metodelor care vor fi explicate în continuare.

```
var calendar = new FullCalendar.Calendar(calendarEl, {
    initialView: 'dayGridMonth',
    selectable: true,
    events: "{{ route('appointments.fetch') }}",

    // Alte configurări
});

calendar.render();
```

Controllerul AppointmentController este cel care se ocupă de logica acestui calendar. Funcția getUnavailableDates() am implementat-o pentru a genera automat lista cu zilele indisponibile din calendarul programărilor. Zilele indisponibile reprezintă zilele din trecut și cele din weekend deoarece centrul de adopție este deschis doar în timpul săptămânii.

Inițial, metoda obține data curentă și generează o perioadă de o lună începând de astăzi, apoi parcurge fiecare zi din interval și verifică dacă este sâmbătă sau duminică. Dacă este cazul, ziua respectivă este adăugată într-un array numit \$weekends, care este returnat ca răspuns în format JSON. Acesta este folosit în partea de front-end pentru a bloca automat din calendar zilele care nu sunt permise pentru programări. Astfel, utilizatorii interacționează doar cu date valide încă din faza de selecție.

```
public function getUnavailableDates()
{
    $today = now();
    $nextMonth = $today->copy()->addMonth();
    $weekends = [];

    for ($date = $today->copy(); $date <= $nextMonth; $date->addDay()) {
        if ($date->isSaturday() || $date->isSunday()) {
            $weekends[] = $date->format('Y-m-d');
        }
    }
    return response()->json($weekends); }
```

Pentru a continua procesul de selecție a programărilor, am implementat și funcția getUnavailableHours, în același controller. Această funcție are rolul de a obține orele deja rezervate de alți utilizatori pentru programări, astfel încât rezultatul să fie transmis în partea de front-end pentru a bloca orele indisponibile.

În primul rând, am interogată tabelul appointments, filtrând rezultatele obținute în funcție de data specificată ca parametru (\$date), pentru a obține toate înregistrările de tipul appointment_date care corespund acelei date. Ulterior, pentru fiecare dată și oră din lista obținută, am extras doar partea care reprezintă ora și minutul, folosind funcția PHP strtotime pentru a transforma data într-un format care poate fi procesat, urmată de date('H:i', ...) pentru a păstra doar ora și minutul. În final, am returnat lista rezultată sub formă de răspuns JSON pentru a putea fi folosit de front-end în procesul de prevenire a utilizatorilor să aleagă ore deja rezervate.

```
public function getUnavailableHours($date)
{
    $bookedHours = DB::table('appointments')
        ->whereDate('appointment_date', $date)
        ->pluck('appointment_date')
        ->map(function ($dateTime) {
            return date('H:i', strtotime($dateTime));
        })
        ->toArray();
    return response()->json($bookedHours); }
```

Pe partea de front-end, am implementat logica de selecție a orei din calendar, în JavaScript, folosindu-mă în principal de metoda getUnavailableHours. Când utilizatorul face click pe o dată din calendar, funcția dateClick verifică dacă acea dată este disponibilă și nu este trecută. Dacă data este validă un modal pentru selectarea orei se deschide, utilizând Bootstrap. Ulterior, se face o cerere către server pentru a obține orele deja rezervate în ziua respectivă. În funcție de răspuns, opțiunile pentru alegerea orei sunt completate dinamic. Orele disponibile sunt afișate între 08:00 și 17:00, iar cele deja rezervate sunt excluse. Dacă nu sunt ore disponibile, utilizatorul va primi un mesaj care indică acest lucru, iar selectarea orei devine imposibilă. Dacă sunt disponibile ore, utilizatorul poate alege o opțiune din listă.

```
dateClick: function(info) {
    let dateStr = info.dateStr;

    if (!unavailableDates.includes(dateStr) && new Date(dateStr) >= new
    Date()) {
        appointmentDate = dateStr;
        document.getElementById('appointment_date').value =
        appointmentDate;
        var appointmentModal = new
        bootstrap.Modal(document.getElementById('appointmentModal'));
        appointmentModal.show();
    }
```

```

        fetch(`/appointments/unavailable-hours/${dateStr}`)
        .then(response => response.json())
        .then(bookedHours => {
            let timeSelect = document.getElementById('appointment_time');
            timeSelect.innerHTML = '<option value="">Alege ora</option>';

            let availableTimes = [];
            let startTime = 8;
            let endTime = 17;

            for (let hour = startTime; hour < endTime; hour++) {
                let timeSlot = `${hour.toString().padStart(2, '0')}:00`;

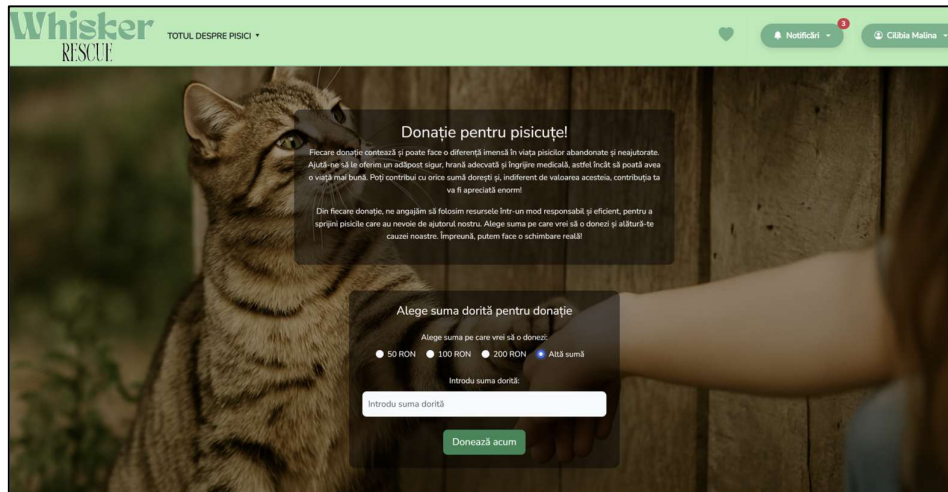
                if (!bookedHours.includes(timeSlot)) {
                    availableTimes.push(timeSlot);
                    let option = document.createElement('option');
                    option.value = timeSlot;
                    option.textContent = timeSlot;
                    timeSelect.appendChild(option);
                }
            }
            if (availableTimes.length === 0) {
                let option = document.createElement('option');
                option.value = "";
                option.textContent = "Nicio oră disponibilă";
                timeSelect.appendChild(option);
                timeSelect.disabled = true;
            } else {
                timeSelect.disabled = false;
            }
        });
    }
}

```

4.2.3 Donații

Sistemul de donații din cadrul aplicației mele este foarte important deoarece asigură susținerea pisicilor care au nevoie de îngrijire. Pentru a realiza acest proces, am dezvoltat o funcționalitate care permite realizarea unei plăți cu cardul prin platforma Stripe. Utilizatorii pot alege o sumă pentru donație, fiecare fiind procesată în timp real și confirmată printr-un mail de mulțumire. În continuare voi detalia componentele esențiale acestei funcționalități.

Când utilizatorul intra pe pagina dedicată donațiilor și dorește să trimită o sumă de bani, acesta are două opțiuni. Poate să doneze o sumă predefinită de 50, 100 sau 200 RON sau poate selecta opțiunea de “Altă sumă”, moment în care apare un câmp suplimentar în care acesta poate introduce ce sumă își dorește să trimită. Acest lucru se poate vizualiza în imaginea de mai jos.



Pentru a implementa această funcționalitate, am creat funcția JavaScript `toggleCustomAmount()`. Funcția este apelată de fiecare dată când utilizatorul selectează o opțiune de sumă. Ea verifică dacă opțiunea aleasă este „other” (adică altă sumă). Dacă da, atunci câmpul de introducere a sumei personalizate este afișat și devine obligatoriu de completat (`required = true`). În caz contrar, câmpul este ascuns și nu mai este necesar la trimiterea formularului.

```
function toggleCustomAmount() {
    const customAmountInput = document.getElementById("custom_amount");
    const customAmountDiv = document.getElementById("customAmountDiv");
    const selectedAmount =
document.querySelector('input[name="amount"]:checked');

    if (selectedAmount && selectedAmount.value === 'other') {
        customAmountDiv.style.display = "block";
        customAmountInput.required = true;
    } else {
        customAmountDiv.style.display = "none";
        customAmountInput.required = false;
    }
}
```

Pentru a integra plata cu Stripe, am creat funcția `createCheckoutSession()` în controllerul `DonationsController`. În primul rând, am preluat suma predefinită sau suma personalizată introdusă de utilizator în formularul din front-end și am convertit-o într-un format de cenți pentru a putea fi utilizată de Stripe în continuare. În continuare, am creat o sesiune de tip “checkout”, în care sunt setate toate detaliile tranzacției: suma, moneda, descrierea

produsului („Donație pentru pisici”), metoda de plată (card) și rutele către care utilizatorul va fi redirecționat în funcție de succesul sau eșecul plății.

După această inițializare, am salvat donația și în baza de date cu statusul inițial pending, care va fi modificat ulterior, când plata va fi îndeplinită. După ce sesiunea a fost creată, utilizatorul este redirecționat către pagina oficială Stripe, unde își poate introduce în siguranță datele cardului pentru a finaliza plata.

```
public function createCheckoutSession(Request $request)
{
    Stripe::setApiKey(env('STRIPE_SECRET'));

    $amount = $request->input('amount');

    if (!$amount) {
        return back()->with('error', 'Te rugăm să alegi o sumă!');
    }

    if ($amount == 'other') {
        $amount = $request->input('custom_amount');
    }

    $amountInCents = (int) ($amount * 100);

    $session = Session::create([
        'payment_method_types' => ['card'],
        'line_items' => [[
            'price_data' => [
                'currency' => 'ron',
                'product_data' => ['name' => 'Donație pentru pisici'],
                'unit_amount' => $amountInCents,
            ],
            'quantity' => 1,
        ]],
        'mode' => 'payment',
        'success_url' => url('/donate/verify/{CHECKOUT_SESSION_ID}'),
        'cancel_url' => route('donation.cancel'),
    ]);

    Donation::create([
        'user_id' => auth()->id(),
        'amount' => $amount,
        'stripe_session_id' => $session->id,
        'status' => 'pending',
    ]);

    return redirect($session->url);
}
```

După ce utilizatorul finalizează plata pe pagina oficială Stripe, funcția `verifyPayment` din același controller verifică dacă tranzacția a fost realizată cu succes. Aceasta funcție primește ca parametru ID-ul sesiunii Stripe și colectează informațiile despre aceasta. Dacă statusul sesiunii este `paid`, înseamnă că plata a fost confirmată. În acest caz, aplicația caută în baza de date donația corespunzătoare acelui ID de sesiune.

Dacă donația există și nu a fost încă marcată ca `succeeded`, atunci se actualizează statusul în baza de date, însemnând că tranzacția a fost finalizată cu succes. În plus, utilizatorul primește automat un email de mulțumire, generat prin clasa `DonationReceived`.

În final, utilizatorul este redirecționat către o pagină de confirmare (`donation_success`). În cazul în care plata nu a fost realizată sau ceva nu a funcționat corect, este afișată o pagină alternativă în care se menționează că procesul de donație a fost anulat (`donation_cancel`).

```
public function verifyPayment($sessionId)
{
    Stripe::setApiKey(env('STRIPE_SECRET'));
    $session = Session::retrieve($sessionId);

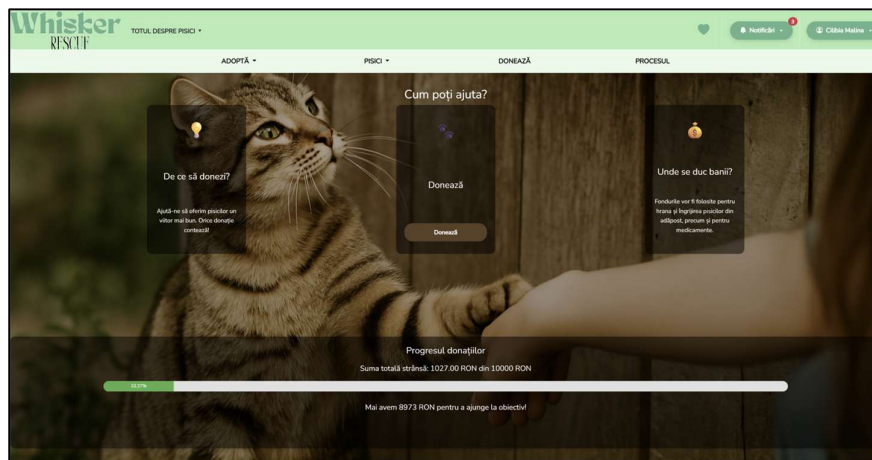
    if ($session->payment_status === 'paid') {
        $donation = Donation::where('stripe_session_id', $sessionId)->first();

        if ($donation && $donation->status !== 'succeeded') {
            $donation->status = 'succeeded';
            $donation->save();

            Mail::to($donation->user->email)->send(new DonationReceived($donation));
        }

        return view('donations.donation_success');
    } else {
        return view('donations.donation_cancel');
    }
}
```

Pentru a încuraja oamenii să realizeze cât mai multe donații, am adăugat în pagina principală de donații și o bară de progres. Această bară arată care este suma de donații care s-a strâns până în momentul de față, cât reprezintă aceasta din obiectivul stabilit de 10 000 RON și câți bani ar mai trebui să se strângă pentru a ajunge la acest obiectiv. Această bară de progres poate fi vizualizată în imaginea de mai jos.



Funcția care se ocupă de bara de progres este `showDonationForm()` din controllerul `DonationsController`. În primul rând am definit suma țintă ($\$targetAmount = 10000$) de 10.000 RON. Apoi, am calculat suma deja donată de utilizatori, folosind modelul `Donation`, luând în considerare doar donațiile care au fost finalizate cu succes (`status = 'succeeded'`).

În continuare am calculat progresul, exprimat ca procent, împărțind suma adunată la suma țintă și înmulțind cu 100. În final, datele (`totalDonations`, `targetAmount`, `progress`) sunt trimise către view-ul `donations.help`, care le afișează în interfața utilizatorului.

```
public function showDonationForm()
{
    $targetAmount = 10000;

    $totalDonations = Donation::where('status', 'succeeded')->sum('amount');

    $progress = ($totalDonations / $targetAmount) * 100;

    return view('donations.help', compact('totalDonations', 'targetAmount', 'progress'));
}
```

4.2.4 Povești de adopție

Secțiunea de povești de adopție este importantă pentru aplicația mea deoarece contribuie la construirea încrederii între utilizatori și platformă. După cum se poate observa și în imaginea de mai jos, în această parte a site-ului, utilizatorii pot descoperi experiențele altor oameni care au adoptat pisici și de asemenea, apăsând butonul de “Adaugă o poveste nouă” pot adăuga și ei la rândul lor propria lor experiență.



Pentru a gestiona trimiterea unei povești de adopție nouă, am creat funcția `store()` în controllerul `AdoptionStoryController`. În această funcție, pentru început, am realizat validarea datelor primite din formularul de adăugare pentru a asigura că titlul și conținutul poveștii au fost completate corect, iar imaginea atașată (dacă există) respectă limitele de dimensiune.

Dacă utilizatorul a încărcat o imagine, aceasta este salvată automat într-un folder numit `stories` din directorul public al aplicației. Apoi, am creat o nouă înregistrare în baza de date în tabela `adoption_stories`, unde am salvat ID-ul utilizatorului care a trimis povestea, restul datelor completate și statusul inițial de `pending` care indică că povestea așteaptă aprobarea unui administrator înainte de a fi afișată public. La final, utilizatorul este redirecționat la pagina “Poveștile mele” (`my.stories`).

```
public function store(Request $request)
{
    $request->validate([
        'title' => 'required|string|max:255',
        'content' => 'required|string',
        'image' => 'nullable|image|max:2048',
    ]);
    $imagePath = null;
    if ($request->hasFile('image')) {
        $imagePath = $request->file('image')->store('stories', 'public');
    }
    AdoptionStory::create([
        'user_id' => Auth::id(),
        'title' => $request->title,
        'content' => $request->input('content'),
        'image' => $imagePath,
        'status' => 'pending',
    ]);
    return redirect()->route('my.stories')->with('success', 'Povestea ta a fost trimisă și
    așteaptă aprobarea.');
```

4.3 Interfața administratorilor

Pentru a asigura o gestionare eficientă a întregii platforme, am creat un panou de administrare dedicat doar utilizatorilor cu rol de administrator. Atunci când un administrator se autentifică pe site, este redirecționat automat pe această pagină unde poate gestiona programările și cererile de adopție, vizualiza statistici relevante, adăuga noi pisici spre adopție și vizualiza utilizatorii site-ului.

4.3.1 Statistici

Prima pagină pe care o vizualizează un administrator este cea dedicată statisticilor. După cum se poate observa și în imaginea de mai jos, această pagină oferă o privire de ansamblu asupra activității din platformă. Administratorul are acces rapid la cele mai relevante informații cum ar fi numărul cererilor de adopție și programărilor săptămânale, evoluția lunară a adopțiilor, a utilizatorilor noi sau a pisicilor adoptate. Fiecare indicator este prezentat sub formă de grafic interactiv permițând analiza datelor într-un mod vizual.



Pentru a genera graficul care afișează evoluția programărilor lunare din cadrul dashboard-ului administrativ, am creat funcția `index()` în controllerul `AdminController`. Pentru început, am agregat datele din tabela `appointments` pe fiecare lună a anului curent folosind o interogare directă prin constructorul de query-uri oferit de Laravel.

Interogarea selectează luna din câmpul `created_at` și numără câte programări au fost înregistrate în fiecare lună, doar pentru anul curent. Rezultatul este stocat într-un array asociativ unde cheia este luna, iar valoarea este numărul de programări. Deoarece nu toate lunile pot conține date, am parcurs toate cele 12 luni cu un `foreach` și am completat valorile lipsă cu 0, pentru a mă asigura că graficul va afișa corect întregul an. Array-ul rezultat, denumit

\$monthlyAppointments, este transmis către view și folosit pentru a genera un grafic de tip linie cu ajutorul bibliotecii Chart.js. În final, pentru a construi graficele care arată distribuția programărilor în săptămâna curentă, am creat două array-uri simple, denumite \$adoptionData și \$appointmentData.

```
$appointmentsByMonth = DB::table('appointments')
->selectRaw('MONTH(created_at) as month, COUNT(*) as total')
->whereYear('created_at', Carbon::now()->year)
->groupBy(DB::raw('MONTH(created_at)'))
->orderBy('month')
->get()
->pluck('total', 'month')
->toArray();

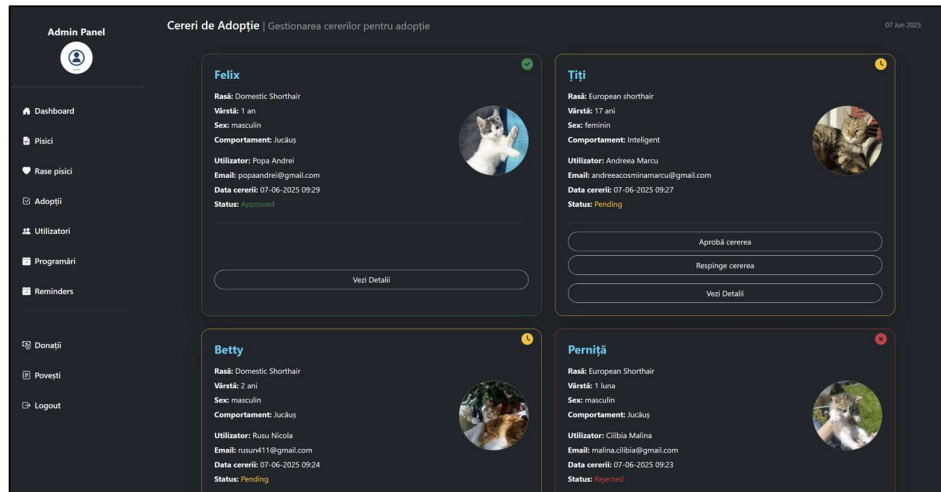
$monthlyAppointments = [];
foreach (range(1, 12) as $month) {
    $monthlyAppointments[] = $appointmentsByMonth[$month] ?? 0;
}

$adoptionData = [
    'approved' => $approved,
    'rejected' => $rejected,
    'pending' => $pending,
];

$appointmentData = [
    'confirmed' => $confirmed,
    'cancelled' => $cancelled,
    'waiting' => $waiting,
];
```

4.3.2 Gestionarea cererilor și a programărilor

Pentru a permite un control mai bun asupra cererilor de adopție, am implementat o pagină dedicată în panoul de administrare. Această pagină oferă vizualizarea fiecărei cereri de adopție, iar administratorul poate vedea informații despre pisică, utilizatorul solicitant și statusul actual. După ce acesta analizează o cerere, are posibilitatea de a o aproba sau respinge, toate aceste acțiuni fiind disponibile direct din interfață, după cum se poate vedea și în imaginea de mai jos.



Pentru procesarea cererilor de adopție, am implementat două funcții esențiale: `approve()` și `reject()`. În cazul aprobării, statusul cererii este actualizat în baza de date ca „approved”, iar orice motiv de respingere anterior este șters. În cazul respingerii, funcția validează mai întâi existența unui motiv (`rejection_reason`), apoi setează statusul cererii ca „rejected” și salvează motivul înregistrat de administrator. După actualizarea cererii, utilizatorul este redirecționat înapoi către pagina principală de administrare a cererilor, împreună cu un mesaj de confirmare.

```
public function approve($id)
{
    $cerere = AdoptionRequest::with('user', 'post')->findOrFail($id);
    $cerere->application_status = 'approved';
    $cerere->rejection_reason = null;
    $cerere->save();

    ....

    return redirect()->route('admin.adoptie')->with('success', 'Cererea a fost aprobată cu succes!');
}

public function reject(Request $request, $id)
{
    $request->validate([
        'rejection_reason' => 'required|string|max:255',
    ]);

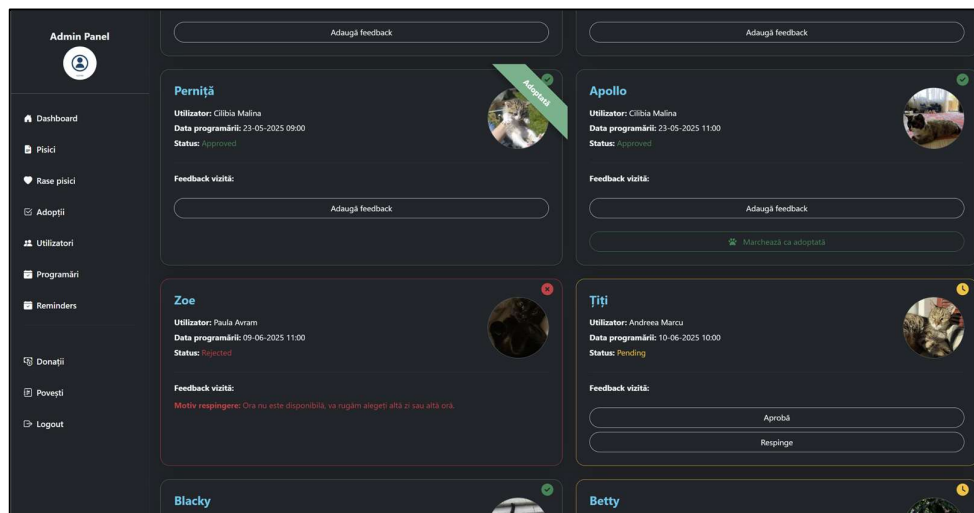
    $cerere = AdoptionRequest::with('user', 'post')->findOrFail($id);
    $cerere->application_status = 'rejected';
    $cerere->rejection_reason = $request->input('rejection_reason');
    $cerere->save();

    ....
}
```



```
return redirect()->route('admin.adoptie')->with('success', 'Cererea a fost respinsă cu succes!');
}
```

Asemănător cu pagina de gestionare a cererilor de adopție, am creat și o pagina pentru gestionarea programărilor. La fel ca și în cazul cererilor, administratorul are posibilitatea de a aproba sau respinge o programare, folosind o logică similară celei descrise anterior. În plus, în această pagină am mai implementat două funcționalități, posibilitatea de a adăuga feedback după finalizarea programării și opțiunea de a marca pisica drept adoptată, în cazul în care utilizatorul a finalizat cu succes procesul de adopție. Această pagina poate fi vizualizată în imaginea de mai jos.



Pentru a permite administratorului să marcheze oficial o pisică ca adoptată, am creat funcția `markAsAdopted()` în controllerul pentru gestionarea programărilor, `AppointmentController`. În această funcție, în primul rând am căutat în baza de date pisica folosind metoda `findOrFail($post_id)`, am setat atributul `adopted` la valoarea `true` și am salvat modificarea. Ulterior, am identificat ultima programare aprobată asociată acelei pisici folosind metoda `latest('appointment_date')` și, dacă aceasta există și are un utilizator valid, am marcat programarea ca fiind finalizată. În acest caz, se trimite automat și un mail cu certificatul de adopție către utilizator, care conține certificatul de adopție sub formă de atașament PDF, folosind clasa `AdoptionCertificateMail`. La final, sistemul redirecționează înapoi către pagină cu un mesaj de confirmare, oferind o modalitate rapidă și completă de încheiere a procesului de adopție.

```
public function markAsAdopted(Request $request, $post_id)
{
    $post = Post::findOrFail($post_id);
    $post->adopted = true;
    $post->save();

    $appointment = \App\Models\Appointment::where('post_id', $post_id)
        ->where('status', 'approved')
        ->latest('appointment_date')
        ->first();

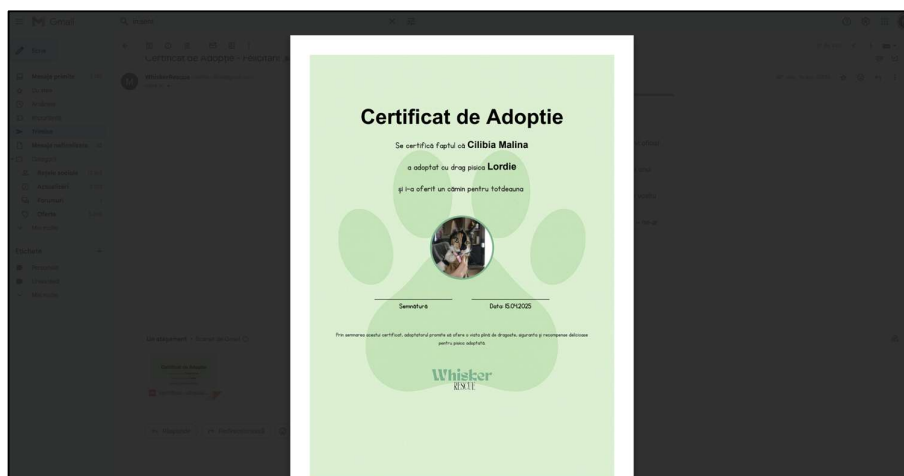
    if ($appointment && $appointment->user) {
        $appointment->completed = true;
        $appointment->save();
        Mail::to($appointment->user->email)->send(new
        AdoptionCertificateMail($appointment->user, $post));
    }
    return back()->with('success', 'Pisica a fost marcată ca adoptată!');
}
```

Pentru a trimite automat certificatul de adopție către utilizator, am creat clasa `AdoptionCertificateMail`, care generează un mail personalizat și atașează un fișier PDF. În metoda `build()`, am folosit `Pdf::loadView()` pentru a crea documentul pe baza view-ului `pdf.certificate`, care conține datele despre utilizator și pisica adoptată. Acest PDF este atașat la mail cu denumirea `certificat-adopție.pdf`, iar astfel, fiecare utilizator primește o confirmare oficială și simbolică a adopției.

```
public function build()
{
    $pdf = Pdf::loadView('pdf.certificate', [
        'user' => $this->user,
        'post' => $this->post
    ]);

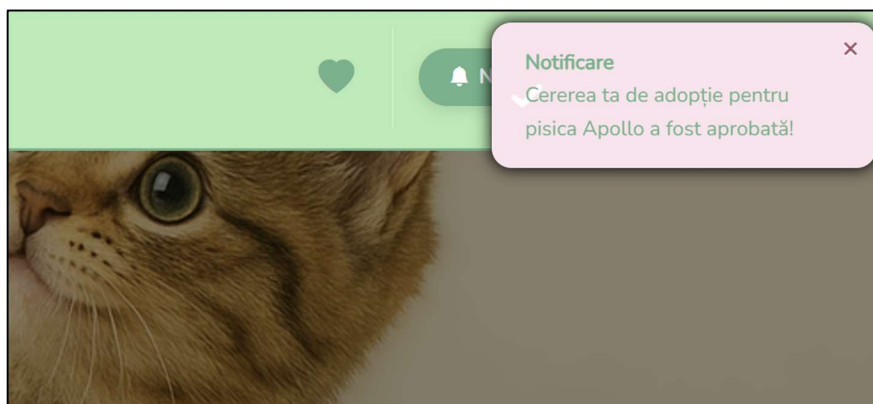
    return $this->subject('Certificat de Adopție - Felicitări! ')
        ->view('emails.adoption.certificate')
        ->attachData($pdf->output(), 'certificat-adopție.pdf', [
            'mime' => 'application/pdf',
        ]); }
}
```

În imaginea de mai jos se poate vedea mailul primit de utilizator după ce a adoptat o pisică, care conține atașat certificatul de adopție în format PDF. Acest document este generat automat cu ajutorul extensiei `barryvdh/laravel-dompdf`, care transformă un view Blade într-un fișier PDF. Certificatul include numele adoptatorului, numele pisicii și data adopției.



4.3.3 Notificări

Pentru a menține o comunicare constantă între utilizator și administrator, am implementat un sistem de notificări în timp real folosind serviciul Pusher. Acest sistem permite transmiterea instantă a mesajelor, precum aprobarea sau respingerea cererilor de adopție, a programărilor, a poveștilor de adopție sau a unui reminder. Notificările apar sub formă de pop-up în interfața utilizatorului, oferindu-i acestuia informații clare, fără a fi nevoie să reîncarce pagina, după cum se poate observa și în imaginea de mai jos.



Pentru a integra aceste notificări, am inclus în funcțiile `approve()` și `reject()` din controllerele administratorilor două linii esențiale, `$user->notify(...)` și `event(...)`. Prima trimite o notificare personalizată către utilizator prin clasa `PushUpNotification`, care poate fi salvată în baza de date, iar cea de-a doua declanșează un eveniment de tip broadcast, precum `AdoptionStatusUpdated`, ce permite afișarea notificării în timp real prin Pusher, fără reîncărcarea paginii.

```
$user->notify(new PushUpNotification(
    "Cererea ta pentru pisica {$post->title} a fost aprobată!",
    url('/home')
));
event(new AdoptionStatusUpdated($user->id, $post->title));
```

Clasa `AdoptionStatusUpdated` este un eveniment care implementează interfața `ShouldBroadcast`, permițând transmiterea notificărilor în timp real prin Pusher. Aceasta primește în constructor ID-ul utilizatorului și titlul postării (pisicii), pe care le stochează ca proprietăți publice. Metoda `broadcastOn()` specifică faptul că notificarea va fi trimisă pe un canal privat, unic pentru fiecare utilizator (`user.{id}`), asigurând astfel confidențialitatea mesajului. În metoda `broadcastWith()`, este definit conținutul efectiv al notificării, sub forma unui mesaj personalizat care menționează numele pisicii adoptate.

```
class AdoptionStatusUpdated implements ShouldBroadcast
{
    public $userId;
    public $postTitle;

    public function __construct($userId, $postTitle)
    {
        $this->userId = $userId;
        $this->postTitle = $postTitle;
    }
    public function broadcastOn()
    {
        return new PrivateChannel('user.' . $this->userId);
    }
    public function broadcastWith()
    {
        return [
            'message' => "Cererea ta de adopție pentru pisica {$this->postTitle} a fost aprobată!"
        ];
    }
}
```

În partea de front-end, am centralizat logica de afișare a notificărilor în timp real folosind JavaScript și Pusher. După ce utilizatorul este abonat la canalul său privat, am definit o funcție `showNotification` care primește tipul notificării, mesajul de afișat și, opțional, un link. La primirea evenimentului `AdoptionStatusUpdated` transmis de back-end, această funcție este apelată și afișează mesajul cu ajutorul bibliotecii `Toastr`. Notificarea redirecționează automat utilizatorul către pagina indicată.

```
if (userId) {
  var channel = pusher.subscribe("private-user." + userId);
  function showNotification(type, message, url = null) {
    var notification = toastr[type]("<strong>Notificare</strong><br>" + message);
    if (url) {
      notification.css('cursor', 'pointer');
      notification.click(() => window.location.href = url);
    }
  }
  channel.bind("App\\Events\\AdoptionStatusUpdated", data => {
    showNotification('success', data.message, data.url);
  });
}
```

Pentru a permite și vizualizarea centralizată a tuturor notificărilor unui utilizator, am creat și un buton în header-ul aplicației, care atunci când este accesat, utilizatorul poate vedea toate notificările pe care le-a primit. Acestea pot fi notificări legate de cereri de adopție, programări, povești de adopție sau remindere pentru programările care au loc ziua următoare. În imaginea de mai jos se poate vizualiza această funcționalitate.



Pentru a realiza această funcționalitate, am creat următoarea funcție JavaScript, în care, atunci când pagina este încărcată complet, am căutat elementul cu ID-ul notifBell, și elementul notifDropdown, adică lista derulantă cu notificările. Când utilizatorul dă click pe clopoțel, comportamentul implicit este blocat, iar meniul de notificări este afișat sau ascuns.

```
document.addEventListener("DOMContentLoaded", function () {
  const bell = document.getElementById("notifBell");
  const dropdown = document.getElementById("notifDropdown");

  bell.addEventListener("click", function (e) {
    e.preventDefault();
    dropdown.classList.toggle("show");
  });
});
```

4.4 Inteligență artificială

Inteligența artificială (AI) reprezintă un domeniu la informaticii care se ocupă cu dezvoltarea de sisteme capabile să simuleze comportamente inteligente, asemănătoare celor umane, precum învățarea, raționamentul, recunoașterea obiectelor sau rezolvarea de probleme. De-a lungul timpului, conceptul de inteligență artificială, a fost interpretat în diferite moduri, ceea ce face dificilă oferirea unei definiții universale. În 1955, John McCarthy, considerat unul dintre fondatorii acestui domeniu, a formulat pentru prima dată o definiție devenită celebră: “Scopul AI este de a dezvolta mașini care se comportă ca și cum ar fi inteligente” [17].

4.4.1 Fundamente teoretice

Rețele neuronale artificiale (ANN)

O rețea neuronală artificială (ANN) este un model matematic care încearcă să simuleze structura și funcționalitățile rețelelor neuronale biologice. Fiecare rețea este alcătuită din neuroni artificiali (unități de calcul simple) care aplică, la nivel de bază, trei operații esențiale: multiplicare, însumare și trecerea rezultatului printr-o funcție de activare [18].

Deși acest mecanism individual este relativ simplu, rețelele devin extrem de puternice atunci când un număr mare de neuroni sunt interconectați în straturi formând arhitecturi complexe. În contextul inteligenței artificiale, rețelele neuronale constituie baza pentru ca sistemele să poată învăța din date. Ele stau la baza unor metode moderne de învățare automată, mai ales în zona de învățare profundă (deep learning). Principalul lor avantaj este ca pot învăța singure din modele și informații, fără a fi necesar să fie programate pas cu pas [18].

Rețele neuronale convoluționale (CNN)

Rețelele neuronale convoluționale (CNN) sunt o extensie a rețelelor neuronale artificiale (ANN) fiind formate tot din neuroni care învață prin ajustarea automată a parametrilor. Ceea ce le diferențiază este arhitectura lor atent proiectată, capabilă să ia în considerare caracteristicile locale și globale ale datelor de intrare. Arhitectura CNN-urilor permite detectarea automată a caracteristicilor locale și globale dintr-o imagine, datorită proprietăților precum conectivitatea locală, invarianța spațială și caracteristicile ierarhice [19].

CNN-urile sunt extrem de eficiente în recunoașterea modelelor vizuale, deoarece pot extrage ierarhic trăsături relevante fără a necesita inginerie manuală a caracteristicilor. Astfel, sunt preferate în sarcini precum clasificarea imaginilor.

MobileNetV2

MobileNetV2 este o arhitectură de rețea neuronală convoluțională (CNN) optimizată special pentru aplicații mobile. A fost dezvoltată de cercetătorii Google ca o versiune îmbunătățită a modelului original MobileNet, având ca obiectiv principal găsirea unui echilibru eficient între dimensiunea modelului și acuratețea sa, un aspect foarte important pentru rularea pe dispozitive cu resurse limitate [20].

Utilizarea MobileNetV2 pentru clasificarea imaginilor oferă mai multe avantaje. Unul dintre principalele avantaje este eficiența sa computațională, care permite rularea rapidă fără a compromite precizia. În comparație cu modele mai complexe și mai mari, arhitectura MobileNetV2 atinge un nivel de performanță similar în sarcini de clasificare a imaginilor, menținând o dimensiune redusă a modelului și timpi de interfață scăzuți.

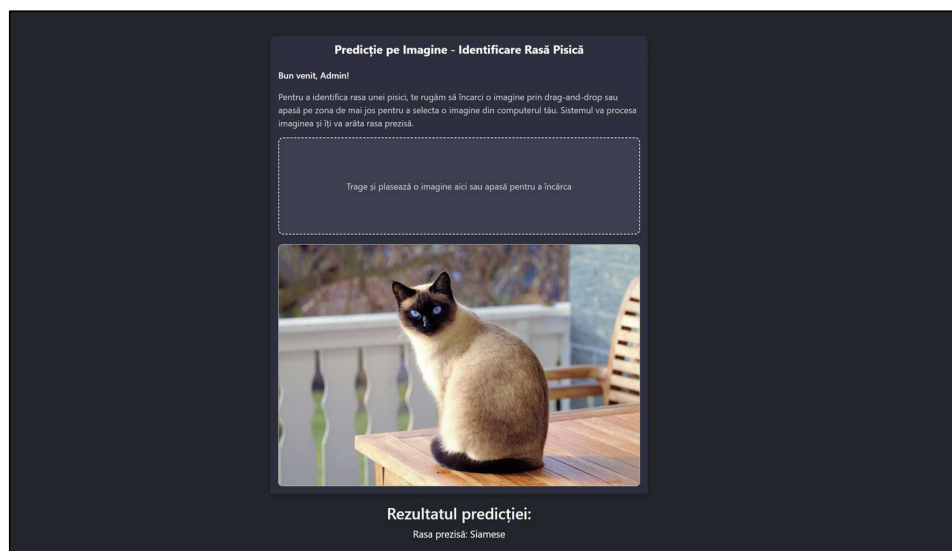
Arhitectura se bazează pe un concept numit blocuri reziduale inversate unde conexiunile reziduale se află între straturile de blocaj. Stratul de expansiune din aceste blocuri utilizează convoluții separabile în adâncime, contribuind astfel la extragerea caracteristicilor relevante într-un mod eficient din punct de vedere computațional [21].

4.4.2 Implementarea componentelor AI în aplicație

Am considerat că integrarea inteligenței artificiale în aplicație este un pas important, deoarece aduce un plus de valoare și oferă o funcționalitate care nu ar fi fost posibilă prin alte metode. Funcționalitatea AI pe care am implementat-o permite identificarea automată a rasei unei pisici, pornind de la o imagine încărcată de utilizator. Am gândit această opțiune în special pentru administratorii platformei, care au responsabilitatea de a adăuga pisici disponibile pentru adopție, dar care nu întotdeauna cunosc rasa exactă a animalului. Prin această soluție, procesul de completare a informațiilor devine mai simplu.

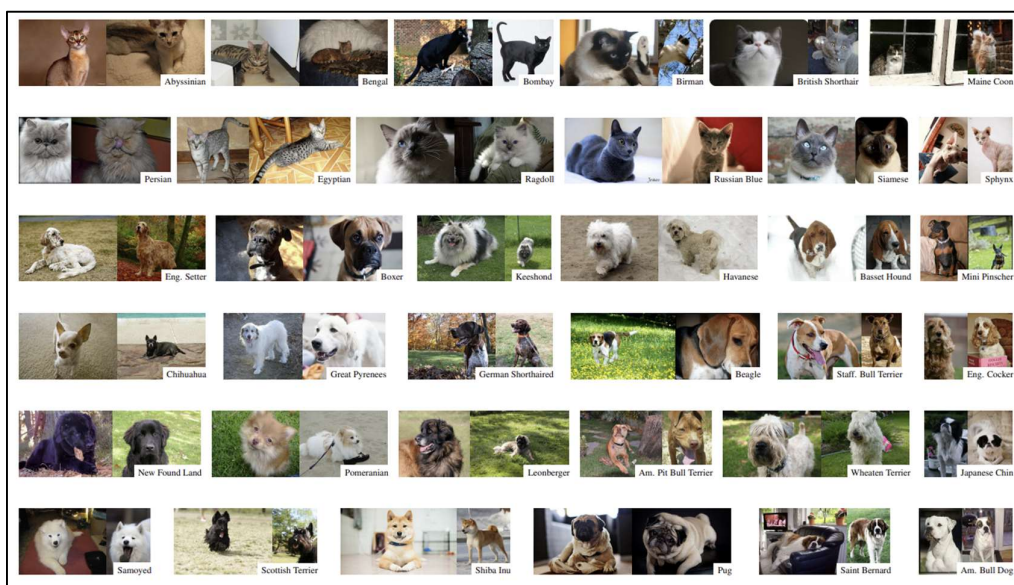
În panoul dedicat administratorilor, am creat o pagină specială unde aceștia pot încărca o imagine prin funcționalitatea drag-and-drop, iar sistemul procesează instant fișierul și afișează sub imagine rezultatul predicției rasei. În imaginea de mai jos, imaginea încărcată a fost recunoscută ca aparținând unei pisici din rasa “Siamese”.

Am realizat întregul proces de dezvoltare și antrenare al modelului AI în mediul Google Colab, care mi-a oferit resursele necesare pentru procesarea imaginilor, antrenarea modelului și generarea fișierului final pentru integrarea în aplicație.



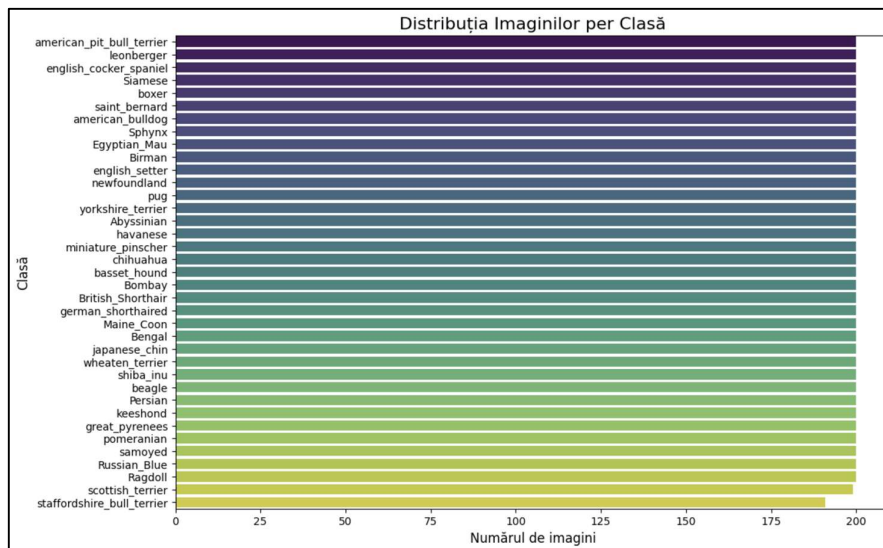
4.4.3 Pregătirea setului de date

Pentru a putea implementa această funcționalitate de recunoaștere a rasei pisicii, primul pas a fost alegerea unui set de date adecvat. Am ales setul de date public Oxford-IIIT Pet Dataset, disponibil pe platforma Kaggle [22]. Acesta conține în total 37 de categorii, care includ atât rase de pisici, cât și de câini, fiecare clasă având aproximativ 200 de imagini. După cum se poate observa și în figura 15, fiecare imagine din acest set vine cu o etichetă care reflectă rasa animalului. Setul de date este organizat în mod clar, permițând identificarea rapidă a fiecărei rase.



Figură 15. Exemple de imagini din Oxford-IIIT Pet Dataset [23]

Pentru a înțelege mai bine structura setului de date și a verifica dacă distribuția imaginilor pe clase este echilibrată, am creat un script care extrage denumirea rasei direct din numele fiecărui fișier imagine. Ulterior, am utilizat biblioteca matplotlib împreună cu seaborn pentru a genera un grafic care ilustrează numărul de imagini pentru fiecare clasă. Această vizualizare m-a ajutat să observ rapid dacă există clase suprasaturate, ceea ce ar fi putut influența negativ acuratețea modelului în la antrenare. După cum se poate observa în figura 16, distribuția este una echilibrată, fiecare clasă având aproximativ 200 de exemple.



Figură 16. Numărul de imagini per clasă în setul de date

Pentru a putea antrena modelul de inteligență artificială, am avut nevoie pentru început de o metodă care să îmi permită să încarc imaginile în loturi (batch-uri), fără a le ține pe toate simultan în memorie. De aceea, am creat un generator personalizat, folosind clasa Sequence din Keras. Acest generator parcurge toate imaginile dintr-un folder, le redimensionează automat la dimensiunea cerută de model (224x224), le normalizează și extrage eticheta clasei din numele fișierului.

Am împărțit apoi aceste imagini în două seturi: unul pentru antrenare și unul pentru testare, păstrând distribuția proporțională a claselor. Am considerat că această împărțire este esențială pentru a putea evalua corect performanța modelului și pentru a preveni supraînvățarea. Generatorul creat returnează pentru fiecare lot un set de imagini și etichetele corespunzătoare, în formatul one-hot encoding, care este necesar pentru funcția categorical_crossentropy folosită la antrenare. În plus, am implementat și o metodă care reamestecă fișierele la finalul fiecărei epoci, astfel încât modelul să nu învețe în funcție de ordinea imaginilor. Următorul cod reprezintă metodele explicate mai sus.

```

import os
import numpy as np
from tensorflow.keras.preprocessing.image import load_img, img_to_array
from tensorflow.keras.utils import to_categorical, Sequence
from sklearn.model_selection import train_test_split

class CustomImageGenerator(Sequence):
    def __init__(self, image_dir, batch_size=32, target_size=(224, 224), num_classes=37,
is_test=False):
        self.image_dir = image_dir
        self.batch_size = batch_size
        self.target_size = target_size
        self.num_classes = num_classes
        self.is_test = is_test

        self.image_files = [f for f in os.listdir(image_dir) if f.endswith('.jpg')]

        self.class_names = [f.rsplit('_', 1)[0] for f in self.image_files]
        self.class_to_index = {class_name: index for index, class_name in
enumerate(sorted(set(self.class_names)))}

        self.train_files, self.test_files = train_test_split(self.image_files, test_size=0.2,
stratify=self.class_names)
        self.files = self.train_files if not self.is_test else self.test_files

    def __len__(self):
        return int(np.floor(len(self.files) / self.batch_size))

    def __getitem__(self, index):
        batch_files = self.files[index * self.batch_size:(index + 1) * self.batch_size]
        images = []
        labels = []

        for file in batch_files:
            img_path = os.path.join(self.image_dir, file)
            img = load_img(img_path, target_size=self.target_size)
            img_array = img_to_array(img) / 255.0
            images.append(img_array)

            class_name = file.rsplit('_', 1)[0]
            class_index = self.class_to_index[class_name]
            labels.append(class_index)

        labels = to_categorical(labels, num_classes=self.num_classes)
        return np.array(images), np.array(labels)

    def on_epoch_end(self):
        np.random.shuffle(self.files)

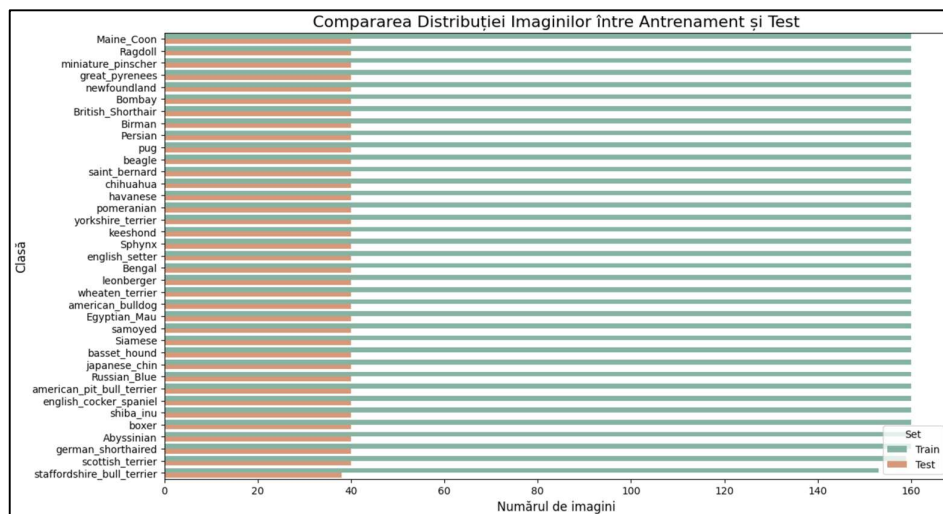
```

```

train_generator = CustomImageGenerator('/kaggle/input/the-oxfordiit-pet-dataset/images',
batch_size=32, num_classes=37, is_test=False)
test_generator = CustomImageGenerator('/kaggle/input/the-oxfordiit-pet-dataset/images',
batch_size=32, num_classes=37, is_test=True)

print("Training batches:", len(train_generator))
print("Test batches:", len(test_generator))
    
```

Pentru a verifica dacă împărțirea imaginilor între setul de antrenament și cel de test a păstrat proporțiile inițiale ale claselor, am creat un script care calculează și compară distribuția fiecărei rase în cele două subseturi. Mai exact, am extras denumirea clasei din fiecare fișier imagine și am numărat câte exemple există pentru fiecare clasă, separat pentru train și test. Apoi, am reprezentat vizual aceste date folosind biblioteca seaborn, într-un grafic comparativ care evidențiază numărul de imagini per clasă în funcție de set. După cum se poate observa în figura 17, distribuția este echilibrată, ceea ce confirmă că stratificarea a fost realizată corect și că modelul va avea șanse egale să învețe din toate clasele.



Figură 17. Compararea distribuției imaginilor între seturile de antrenament și test

4.4.4 Antrenarea și evaluarea modelului

După pregătirea setului de date și definirea generatorului personalizat, următorul pas a fost antrenarea efectivă a modelului de inteligență artificială. În această etapă, modelul învață să recunoască rasele de pisici pe baza imaginilor prelucrate anterior. Pe parcursul procesului de antrenare, am evaluat constant performanța modelului pentru a mă asigura ca modelul învață corect imaginile și nu doar memorează din setul de antrenament.

Pentru antrenarea modelului de clasificare, am folosit arhitectura MobileNetV2, pe care am încărcat-o fără stratul final. Am vrut să folosesc baza antrenată deja pe un set complex de imagini și să o adaptez la nevoile aplicației mele. Am aplicat fine-tuning, permițând actualizarea greutateților întregului model în timpul antrenării, pentru a obține rezultate mai bune. Peste baza MobileNetV2 am adăugat un strat de global average pooling pentru a reduce dimensiunea ieșirii, urmat de un strat dens cu 1024 de neuroni, un strat de Dropout (0.5) pentru a preveni overfitting-ul și un strat final cu activare softmax, necesar pentru clasificarea celor 37 de rase.

Am compilat modelul folosind optimizatorul Adam, cu o rată de învățare mica, de 0.00001, tocmai pentru a permite un fine-tuning al greutateților. Funcția de pierdere pe care am ales-o a fost categorical_crossentropy, potrivită pentru o problemă de clasificare multi-clasă. Am antrenat modelul timp de 30 de epoci, iar pe parcurs am evaluat constant atât acuratețea, cât și evoluția valorii loss pe seturile de antrenament și validare. Acest proces m-a ajutat să monitorizez învățarea și să observ semne de supraînvățare. Codul de mai jos reprezintă procesul de învățare.

```
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.layers import Dropout

base_model = tf.keras.applications.MobileNetV2(
    include_top=False,
    weights='imagenet',
    input_shape=(224, 224, 3)
)

model = tf.keras.Sequential([
    base_model,
    tf.keras.layers.GlobalAveragePooling2D(),
    tf.keras.layers.Dense(1024, activation='relu'),
    Dropout(0.5),
    tf.keras.layers.Dense(train_generator.num_classes, activation='softmax')
])

model.compile(optimizer=Adam(learning_rate=0.00001), loss='categorical_crossentropy',
metrics=['accuracy'])

for layer in model.layers:
    print(f'Layer {layer.name} trainable: {layer.trainable}')

history = model.fit(
    train_generator,
    steps_per_epoch=len(train_generator),
```

```
epochs=30,
validation_data=test_generator,
validation_steps=len(test_generator)
)
```

După finalizarea celor 30 de epoci de antrenament, am analizat evoluția performanței modelului folosindu-mă atât de valorile acurateții și pierderii, cât și de graficele și tabelul generat automat de Keras. Începând cu epoca 1, unde modelul avea o acuratețe scăzută (doar 5.3%), acesta a învățat treptat din datele de antrenament, ajungând la o acuratețe de aproape 99% până în epoca 30. Pentru procesul de evaluare la fel de importantă este și urmărirea acurateții pe setul de validare, care a crescut în mod constant, atingând valoarea de 97.8%, ceea ce înseamnă că modelul nu doar a memorat imaginile, ci a reușit să generalizeze foarte bine pe date noi.

Pentru a confirma aceste rezultate, am rulat și o evaluare finală a modelului pe întregul set de test, folosind funcția `model.evaluate(test_generator)`. În urma acestei evaluări, am obținut o pierdere de aproximativ 0.0723 și o acuratețe finală de 97.82%, la fel cum reiese și din ultimele epoci din tabel. Aceste rezultate mi-au arătat că modelul funcționează foarte bine și că alegerea arhitecturii MobileNetV2 și strategiile de fine-tuning aplicate au fost decizii potrivite pentru problema mea de clasificare a rasei pisicii. Rezultatele obținute de model pe parcursul celor 30 de epoci se pot observa în tabelul 1.

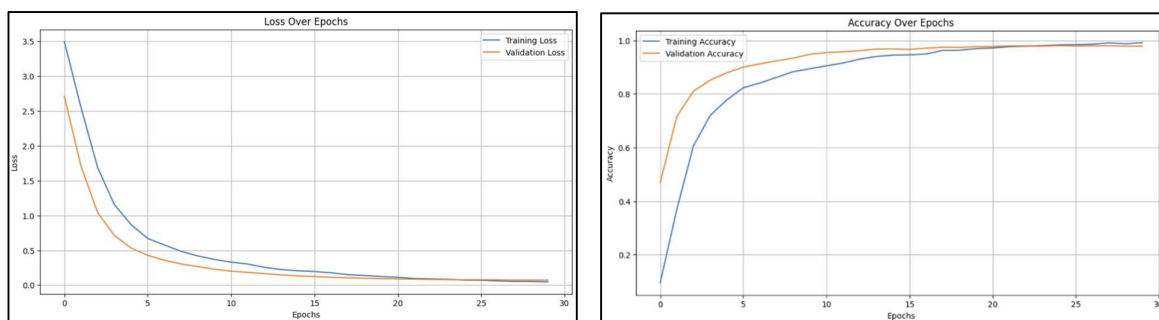
Tabel 1. Rezultatele obținute în timpul antrenării și validării modelului AI

<i>Epocă</i>	<i>Acuratețea pe setul de antrenament</i>	<i>Pierdere pe setul de antrenament</i>	<i>Acuratețea pe setul de validare</i>	<i>Pierdere pe setul de validare</i>
1	0.0534	3.7422	0.4681	2.7103
2	0.2878	2.7850	0.7154	1.7080
3	0.5716	1.8470	0.8098	1.0405
4	0.6997	1.2661	0.8512	0.7132
5	0.7643	0.9214	0.8784	0.5333
6	0.8174	0.6841	0.9001	0.4273
7	0.8421	0.5815	0.9124	0.3566
8	0.8563	0.5050	0.9232	0.3031
9	0.8861	0.4262	0.9334	0.2662
10	0.8873	0.3763	0.9477	0.2278
11	0.9055	0.3320	0.9545	0.2001
12	0.9168	0.3007	0.9572	0.1827
...	...	...	...	...
29	0.9893	0.0507	0.9783	0.0731
30	0.9896	0.0489	0.9783	0.0723

Pentru a avea și o vizualizare grafică a modelului în timpul antrenării, am generat două grafice, unul pentru pierdere și altul pentru acuratețe, care se regăsesc în figura 18. Scopul acestor grafice a fost să văd dacă modelul învață corect și dacă nu începe să memoreze datele, în loc să le generalizeze.

În primul grafic, cel cu pierderea (Loss Over Epochs), se poate observa că valorile scad constant, atât pe setul de antrenament, cât și pe cel de validare. Asta confirmă faptul că modelul învață treptat, iar erorile devin tot mai mici.

În al doilea grafic, cel cu acuratețea (Accuracy Over Epochs), curbele cresc constant și ajung aproape de 100%, ceea ce demonstrează faptul că rețeaua reușește să recunoască imaginile tot mai bine, fără diferențe mari între antrenament și validare. Astfel, se poate observa că modelul nu a nu a supraînvățat și chiar funcționează bine și pe imagini noi.



Figură 18. Evoluția pierderii și acurateții modelului în timpul antrenării

4.4.5 Integrarea modelului în aplicație

După ce am antrenat și salvat modelul în Google Colab, a fost necesar să îl convertesc într-un format compatibil cu TensorFlow.js, astfel încât să îl pot folosi direct în browser în aplicația mea. Am realizat această conversie folosind comanda `tensorflowjs_converter`, care transformă modelul salvat în format SavedModel într-un model .json cu fișiere .bin, compatibil cu Laravel.

```
tf.saved_model.save(model, 'sample_data/tf_model')

!tensorflowjs_converter --input_format=tf_saved_model --output_format=tfjs_graph_model
sample_data/tf_model/ sample_data/tfjs_model
```

După conversie, am adăugat fișierele cu extensiile .json și .bin obținute într-un folder numit /models în Laravel. Apoi, în dashboard-ul pentru administratori, am creat o pagina

dedicată predicțiilor unde administratorul poate posta o imagine, iar modelul încarcă imaginea, o procesează și returnează numele rasei prezise.

În partea de JavaScript a aplicației, am folosit librăria `@tensorflow/tfjs`, pentru a încărca modelul cu `tf.loadGraphModel()` și realiza predicția direct în browser. Am realizat o funcție `predictImage()` care preia imaginea introdusă de utilizator, o transformă într-un tensor, o redimensionează la 224x224, o normalizează și o pregătește pentru a fi procesată. Modelul returnează apoi un vector de predicții, se extrage clasa cu probabilitatea cea mai mare și se afișează numele rasei direct sub imagine. Totul se întâmplă local, fără server, ceea ce face procesul foarte rapid. Codul care face acest lucru este următorul.

```
async function predictImage(image) {
  const model = await loadModel();

  const tensor = tf.browser.fromPixels(image)
    .resizeNearestNeighbor([224, 224])
    .toFloat()
    .div(tf.scalar(255))
    .expandDims(0);

  const prediction = model.predict(tensor);
  prediction.print();

  const predictedClass = prediction.argmax(-1).dataSync()[0];
  const predictedClassName = classNames[predictedClass];

  const predictionText = document.getElementById('predictionText');
  predictionText.innerHTML = `Rasa prezisă: ${predictedClassName}`;
}
```

5. Evaluarea

Acest capitol este dedicat procesului de evaluare a aplicației dezvoltate, prin testare manuală și automată. În cele ce urmează voi analiza buna funcționare a funcționalităților principale ale aplicației web din cadrul acestei lucrări.

5.1 Testarea manuală

„Testarea este procesul de executare a unui program cu intenția de a găsi erori” [24]. În cadrul procesului clasic de dezvoltare software, testarea manuală a reprezentat o componentă esențială pentru identificarea defectelor și verificarea corectitudinii funcționalităților. Această metodă presupune interacțiunea directă a testerului cu aplicația, fără instrumente automate, pentru a valida comportamentul acesteia în diferite scenarii [25].

Pentru testarea funcționalităților dedicate administratorilor am realizat mai multe cazuri de testare manuale în Microsoft Excel. Primul caz de testare, se regăsește în figura 19 și reprezintă verificarea funcționalității de inteligență artificială. Am încărcat o poză cu o pisică siameză în pagina dedicată și am urmărit dacă sistemul afișează corect rasa. Totul a funcționat conform așteptărilor, astfel rezultatul a fost reușit.

ID Caz de testare	AI_002	Descriere caz de testare	Verificarea funcționalității de predicție a rasei pisicii		
Creat de	Cilbia Mălina	Revizuit de	Coordonator	Versiune	1
Numele testerului	Cilbia Mălina	Data testării	9-Iun-2025	Rezultat caz de test (Reușit/Eșuat/Nu s-a executat)	Reușit
S #	Precondiții:	S #	Date de test		
1	Acces la browser cu funcționalitate AI	1	Imagine pisică = siameza.jpg		
2	Acces la pagina de predicție	2			
3		3			
4		4			
Scenariu de test	Verificarea afișării corecte a rasei prezise în urma încărcării unei imagini valide cu o pisică.				
Pas #	Detalii pas	Rezultate așteptate	Rezultate obținute	Reușit / Eșuat / Neexecutat / Suspendat	
1	Navighează la pagina de recunoaștere rasă pisică	Pagina se deschide	Pagina s-a deschis conform așteptărilor	Reușit	
2	Încarcă imaginea siameza.jpg	Imaginea este acceptată și încărcată în interfață	Imaginea a fost încărcată	Reușit	
3	Apasă butonul „Precizare rasă”	Se afișează predicția (ex: „Siameză”)	Predicția afișată: „Siameză”	Reușit	
4	Verifică dacă rezultatul apare în zona de output	Zona de output afișează corect numele rasei prezise	Output afișat cu rasa prezisă	Reușit	

Figură 19. Cazul de testare manuală 1

Următoarea funcționalitate pentru care am realizat un test manual este aprobarea unei cereri de adopție din partea unui admin. Acest test poate fi vizualizat în figura 20. Am urmărit

să verific dacă sistemul actualizează corect statusul cererii în urma aprobării și dacă se trimite automat un mail și o notificare utilizatorului. Totul a funcționat conform așteptărilor, deci testul a fost trecut.

ID Caz de testare	ADOPT_003	Descriere caz de testare	Aprobarea unei cereri de adopție de către admin		
Creat de	Glibia Mălina	Revizuit de	Coordonator	Versiune	1
Numele testerului	Glibia Mălina	Data testării	9-Iun-2025	Rezultat caz de test (Reușit/Eșuat/Nu s-a executat)	Reușit
S #	Precondiții:	S #	Date de test		
1	Existența unei cereri de adopție în așteptare	1	ID cerere: #20		
2	Autentificare ca admin	2	Email admin: malina.glibia@gmail.com		
3	Utilizatorul are email valid și este autentificat	3	Email user: rusun411@gmail.com		
4		4			
Scenariu de test	Verificarea aprobării unei cereri de adopție de către admin, cu schimbarea statusului, trimiterea unui e-mail și generarea unei notificări live către utilizator.				
Pas #	Detalii pas	Rezultate așteptate	Rezultate obținute	Reușit / Eșuat / Neexecutat / Suspendat	
1	Autentificare în panoul adminului	Acces la dashboard admin	Dashboard-ul adminului s-a încărcat	Reușit	
2	Navigare către secțiunea „Cereri de adopție”	Lista cererilor este afișată	Cererea cu ID #20 este prezentă în listă	Reușit	
3	Click pe „Aprobă” pentru cererea #20	Statusul cererii se schimbă în „Aprobat”	Statusul s-a schimbat cu succes	Reușit	
4	Verificare trimitere e-mail către utilizator	Utilizatorul primește un e-mail de confirmare privind aprobarea cererii	Emailul a fost trimis și recepționat de utilizator	Reușit	
	Verificare apariție notificare în contul utilizatorului logat	În contul userului apare o notificare privind aprobarea cererii	Notificarea a fost afișată în dropdown-ul de notificări	Reușit	

Figură 20. Cazul de testare manuală 2

În cazul de testare din figura 21, am verificat dacă un admin poate adăuga cu succes o nouă pisică pe site, inclusiv cu imaginea principală și galeria foto. Am completat toate câmpurile necesare, am încărcat imaginile și am validat că pisica apare în lista finală după salvare. Rezultatul a fost reușit deoarece toți pașii au fost efectuați conform așteptărilor.

ID Caz de testare	CAT_004	Descriere caz de testare	Adăugarea unei pisici în platformă de către admin		
Creat de	Cilibia Mălina	Revizuit de	Coordonator	Versiune	1
Numele testerului	Cilibia Mălina	Data testării	9-Iun-2025	Rezultat caz de test (Reușit/Eșuat/Nu s-a executat)	Reușit
S #	Precondiții:	S #	Date de test		
1	Admin autentificat și cu acces la secțiunea Pisici	1	Nume = Lili, vârstă = 1 an, rasă = Siameză, sex = femelă, comportament = blândă		
2	Formular complet funcțional	2	Imagine principală = lili_cover.jpg, galerie = lili1.jpg, lili2.jpg		
3		3			
4		4			
Scenariu de test	Verificarea procesului de adăugare a unei noi pisici în sistem de către un utilizator cu rol de admin, inclusiv validarea datelor și confirmarea adăugării.				
Pas #	Detalii pas	Rezultate așteptate	Rezultate obținute		Reușit / Eșuat / Neexecutat / Suspendat
1	Autentificare ca admin	Acces la panoul admin	Panoul admin s-a încărcat		Reușit
2	Navigare la secțiunea „Pisici”	Afișare listă cu pisicile existente	Lista afișată		Reușit
3	Click pe „Adaugă pisică nouă”	Se deschide formularul de adăugare	Formularul s-a afișat		Reușit
4	Completare câmpuri text (nume, rasă, vârstă etc.)	Datele sunt acceptate de formular	Datele au fost introduse corect		Reușit
5	Încărcare imagine principală (lili_cover.jpg)	Imaginea se încarcă și se afișează în previzualizare	Imagine afișată corect în zona cover		Reușit
6	Încărcare imagini în galerie (lili1.jpg, lili2.jpg)	Galeria se încarcă și apar miniaturile sub formular	Imaginile apar în „Galerie existentă”		Reușit
7	Click pe „Salvează modificările”	Pisica este adăugată cu succes, mesaj de confirmare vizibil	Mesaj afișat: „Pisica a fost adăugată cu succes”		Reușit
8	Verificare afișare corectă a galeriei după salvare	Galeria este păstrată și imaginile se pot vizualiza	Imaginile sunt afișate în lista finală		Reușit

Figură 21. Cazul de testare manuală 3

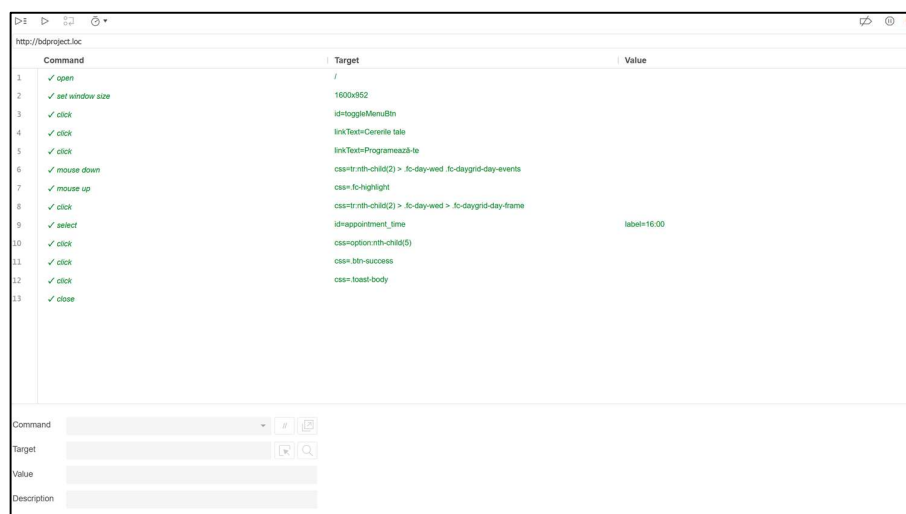
În urma testării manuale realizate pe funcționalitățile administratorului, am ajuns la concluzia ca sistemul răspunde corect și stabil la cereri. Fiecare scenariu testat a funcționat conform așteptărilor, fără erori sau comportamente neprevăzute. Astfel, aplicația este pregătită din punct de vedere funcțional pentru utilizare.

5.2 Testarea automată

În contextul actual al dezvoltării rapide și iterative, testarea exclusiv manuală nu mai poate asigura acoperirea și eficiența necesare, de aceea testarea automată a devenit o practică esențială în dezvoltarea software, contribuind la creșterea vitezei și fiabilității procesului de validare. Aceasta permite rularea repetată a testelor fără intervenție umană, fiind ideală pentru detectarea rapidă a erorilor și asigurarea consistenței în funcționarea aplicației [25].

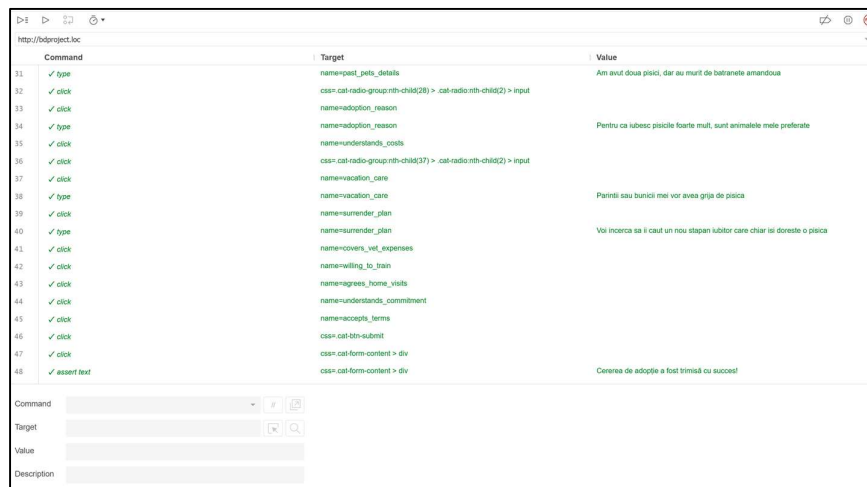
Unul dintre cele mai populare instrumente pentru automatizarea testării aplicațiilor web este Selenium, un cadru open-source care oferă suport pentru mai multe limbaje de programare și browsere. Datorită flexibilității și capacității sale de a gestiona aplicații web complexe, Selenium este frecvent utilizat pentru testarea funcțională și de regresie, contribuind la optimizarea ciclului de viață al aplicației [25].

Pentru a avea un plan complet de testare, pentru testarea funcționalităților utilizatorilor am ales să realizez o testare automată folosind Selenium IDE. Primul test pe care l-am realizat, care se regăsește în figura 22, este pentru a verifica dacă procesul de programare al unei vizite funcționează corect. Scenariul reproduce selectarea unei zile disponibile din calendar, alegerea unei ore și trimiterea cererii. Testul a fost executat cu succes.



Figură 22. Cazul de testare automată 1

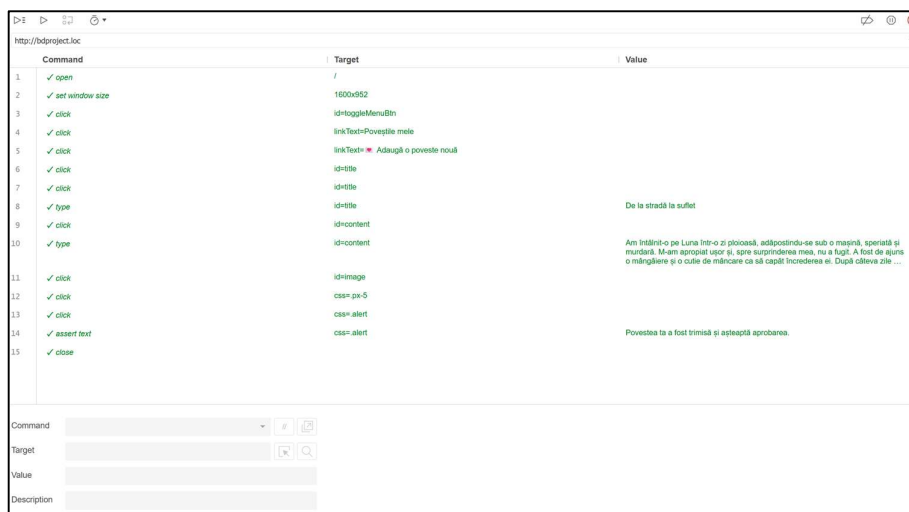
Al doilea caz de testare pe care l-am realizat, după cum se poate observa și în figura 23, este pentru a verifica automat completarea și trimiterea formularului de adopție. Am testat toate câmpurile, cele de date personale și întrebările despre experiența cu animalele pentru a mă asigura că sunt completate corect și că formularul este validat și trimis cu succes. Testul a fost executat fără erori, astfel logica de validare și salvare funcționează conform așteptărilor.



Command	Target	Value
✓ type	name=past_pets_details	Am avut doua pisici, dar au murit de batranete amandoua
✓ click	css=cat-radio-group:nth-child(2) > cat-radio:nth-child(2) > input	
✓ click	name=adoption_reason	
✓ type	name=adoption_reason	Pentru ca iubesc pisicile foarte mult, sunt animalele mele preferate
✓ click	name=understands_costs	
✓ click	css=cat-radio-group:nth-child(3) > cat-radio:nth-child(2) > input	
✓ click	name=vacation_care	
✓ type	name=vacation_care	Parinti sau bunici mei vor avea grija de pisica
✓ click	name=surrender_plan	
✓ type	name=surrender_plan	Voi incerca sa ii caut un nou stapan iubitor care chiar si doreste o pisica
✓ click	name=covers_vet_expenses	
✓ click	name=willing_to_train	
✓ click	name=agrees_home_visit	
✓ click	name=understands_commitment	
✓ click	name=accepts_terms	
✓ click	css=cat-btn-submit	
✓ click	css=cat-form-content > div	
✓ assert text	css=cat-form-content > div	Cererea de adopție a fost trimisă cu succes!

Figură 23. Cazul de testare automată 2

În ultimul caz de testare automată pe care l-am realizat, cel din figura 24, am verificat dacă utilizatorul poate adăuga cu succes o poveste de adopție. Scenariul simulează completarea titlului, conținutului și trimiterea formularului. La final, am obținut mesajul de confirmare ca povestea a fost trimisă și așteaptă aprobarea unui administrator. Rezultatul a fost reușit, ceea ce înseamnă că și această funcționalitate decurge conform așteptărilor.



Command	Target	Value
✓ open	/	
✓ set window size	1600x952	
✓ click	id=toggleMenuBtn	
✓ click	linkText=Poveștile mele	
✓ click	linkText= Adaugă o poveste nouă	
✓ click	id=title	
✓ click	id=title	De la stradă la suflet
✓ type	id=content	
✓ click	id=content	
✓ type	id=content	Am întâlnit-o pe Luna într-o zi ploioasă, adăpostindu-se sub o mașină, speriată și murdară. M-am apropiat ușor și, spre surprinderea mea, nu a fugit. A fost de ajuns o mișcare și o cutie de mâncare ca să capteze încrederea ei. După câteva zile...
✓ click	id=image	
✓ click	css=cat-pn-5	
✓ click	css=alert	
✓ assert text	css=alert	Povestea ta a fost trimisă și așteaptă aprobarea.
✓ close		

Figură 24. Cazul de testare automată 3

În urma cazurilor de testare automată am ajuns la concluzia ca și funcționalitățile utilizatorilor funcționează corect. Prin intermediul Selenium IDE am demonstrat corectitudinea proceselor precum completarea unui formular, programarea unei vizite sau adăugarea unei povești. Fiecare test a fost executat în final cu succes, ceea ce confirmă aplicația funcționează conform așteptărilor, fără erori vizibile.

Concluzii

Lucrarea de față a avut ca obiectiv dezvoltarea unei aplicații web dedicate adopției de pisici, cu accent pe automatizarea proceselor deja existente. Într-un context în care multe dintre etapele adopției se desfășoară încă în mod tradițional, mi-am dorit să ofer o soluție digitală completă, care să simplifice interacțiunea dintre utilizatori și centrele de adopție.

Un element central al aplicației este integrarea unui modul de inteligență artificială care permite recunoașterea automată a rasei unei pisici pe baza unei imagini încărcate. Această funcționalitate nu doar că adaugă un plus de valoare aplicației, dar oferă și o experiență interactivă, care poate fi foarte utilă administratorilor în procesul de înregistrare a pisicilor.

Pe lângă acest modul AI, aplicația include funcționalități esențiale precum trimiterea cererilor de adopție, programarea vizitelor, sistemul de notificări în timp real și gestionarea poveștilor de adopție. Toate aceste funcționalități le-am gândit pentru a crea o experiență cât mai prietenoasă pentru utilizator, indiferent dacă acesta este un adoptator nou sau un administrator de adăpost. Interfața aplicației este ușor de folosit, iar fiecare secțiune am construit-o pornind de la nevoile reale observate în procesul tradițional de adopție. După ce am testat fiecare dintre aceste funcții, atât manual, cât și automat, am putut confirma că aplicația funcționează stabil și răspunde corect la interacțiunile utilizatorilor în scenarii reale de utilizare.

Consider că aplicația realizată în această lucrare răspunde nevoii tot mai mari de modernizare a proceselor din domeniul protecției animalelor, contribuind la o experiență de adopție mai prietenoasă, mai rapidă și mai bine organizată. În viitor, aplicația ar putea fi dezvoltată și mai mult prin extinderea componentelor de inteligență artificială, adăugarea unui sistem de recomandare personalizată sau integrarea cu servicii externe de monitorizare post-adopție.

Bibliografie

- [1] S. Sachdeva, „Scrum methodology,” *Int. J. Eng. Comput. Sci*, vol. 5, nr. 16792, pp. 16792--16800, 2016.
- [2] . A. G. Rahaman, V. Gayatri, C. S. Kiran, K. S. Pavan, B. Bhumika și V. Sateesh, „Development of Web Applications by Integrating Frontend and Backend Tools,” *International Journal of Innovative Research in Computer and Communication Engineering*, pp. 5002--5007, 2023.
- [3] J. Spurlock, *Bootstrap: responsive web development*, O'Reilly Media, Inc., 2013.
- [4] H. Marjin, *Eloquent JavaScript*.
- [5] M. Bean, *Laravel 5 essentials*, Packt Publishing Ltd, 2015.
- [6] Vue.js, „Vue.js - The Progressive JavaScript Framework,” [Online]. Disponibil la: <https://vuejs.org/>. [Accesat 25 aprilie 2025].
- [7] O. P. Luzardo, J. E. Zaldívar-Laguía, M. Zumbado și M. D. M. Travieso-Aja, „{The role of veterinarians in managing community cats: a contextualized, comprehensive approach for biodiversity, public health, and animal welfare,” *Animals*, vol. 13, nr. 10, p. 1586, 2023.
- [8] I. Lungu, I. Botha și V. Manole, „Database Systems—Present and Future.,” *Revista Informatica Economică*, vol. 13, nr. 1, 2009.
- [9] Oracle, „What is MySQL?,” Oracle Corporation, [Online]. Disponibil la: <https://www.oracle.com/mysql/what-is-mysql/>. [Accesat 26 aprilie 2025].
- [10] B. Cooksey, *An introduction to APIs*, Zapier, 2014.
- [11] Cygnis Media, „Back-End Development for Custom Web Applications – A Guide,” 2023. [Online]. Disponibil la: <https://cygnis.co/blog/back-end-development-for-custom-web-applications-guide/>. [Accesat 25 aprilie 2025].
- [12] N. Kamel și J.-L. Lanet, „Analysis of HTTP protocol implementation in smart card embedded web server.,” *International Journal of Information and Network Security*, vol. 2, nr. 5, p. 417, 2013.
- [13] R. Eklblom, în *Applied Representational State Transfer.*, 2011.
- [14] M. Laaziri, K. Benmoussa, S. Khouilji, K. M. Larbi și A. El Yamami, în *A comparative study of laravel and symfony PHP frameworks. International Journal of Electrical and Computer Engineering*, vol. 9, 2019, pp. 704-712.
- [15] Z. Subecz, „Web-Development with Laravel Framework,” *Gradus*, vol. 8, nr. 1, pp. 211-218, 2021.

- [16] N. Sahu, „How backend and frontend connect and work together in web development,” 4 iulie 2024. [Online]. Disponibil la: <https://nikhilsomansahu.medium.com/how-backend-and-frontend-connect-and-work-together-in-web-development-2c852bd1e9e6>. [Accesat 26 aprilie 2025].
- [17] W. Ertel, Introduction to artificial intelligence, Springer Nature, 2024.
- [18] A. Krenker, B. Janez și K. Andrej , „Introduction to the artificial neural networks,” *Artificial Neural Networks: Methodological Advances and Biomedical Applications. InTech*, pp. 1-18, 2011.
- [19] K. O'shea și R. Nash, „An introduction to convolutional neural networks,” *arXiv preprint arXiv:1511.08458*, 2015.
- [20] M. Sandler, A. Howard , M. Zhu , A. Zhmoginov și L.-C. Chen, „MobileNetV2: Inverted Residuals and Linear Bottlenecks,” 2019.
- [21] Analytics Vidhya, „What is MobileNetV2?,” Decembrie 2023. [Online]. Disponibil la: <https://www.analyticsvidhya.com/blog/2023/12/what-is-mobilenetv2/>. [Accesat 27 Aprilie 2025].
- [22] Kaggle, „The Oxford-IIIT Pet Dataset,” 2022. [Online]. Disponibil la: <https://www.kaggle.com/datasets/tanlikesmath/the-oxfordiiit-pet-dataset/data>. [Accesat aprilie 14 2025].
- [23] O. M. Parkhi, A. Vedaldi, A. Zisserman și C. V. Jawahar, „IEEE Conference on Computer Vision and Pattern Recognition (CVPR),” în *Cats and Dogs*, Providence, Rhode Island, SUA, 2012.
- [24] Myers, Glenford J, Sandler, Corey , Badgett și Tom, The art of software testing, John Wiley & Sons, 2011.
- [25] Jayaraman, Kumaresan Durvas și Er Aman Shrivastav, „Automated Testing Frameworks: A Case Study Using Selenium and NUnit,” *International Journal of Research in Humanities & Social Sciences*, vol. 13, nr. 1, pp. 1--16, 2025.

Anexe

Anexa 1. Chestionar pentru stabilirea funcționalităților necesare site-urilor de adopție de pisici. Chestionarul este disponibil la link-ul:

<https://docs.google.com/forms/d/e/1FAIpQLSc60wumur27o9cktsWZGfyyJDRTi2znIKQgJUowGkqcCbR-kA/viewform?usp=dialog>

1. Ți-ai imaginat vreodată că ai putea adopta o pisică?
 - Da, m-am gândit la asta
 - Nu, dar nu exclud ideea
 - Nu, nu sunt interesat(ă)
2. Din punctul tău de vedere, care sunt cele mai frecvente motive pentru care oamenii aleg să nu adopte pisici în România?
 - Lipsa de informare despre cum se adoptă
 - Frica de responsabilitate pe termen lung
 - Costurile asociate (hrană, tratamente, sterilizare)
 - Procedurile de adopție sunt complicate sau neclare
 - Nu există o platformă centralizată și accesibilă
 - Lipsa unui proces digitalizat
3. Dacă ai vrea să adopți o pisică, ce ți s-ar părea cel mai dificil în procesul de adopție?
 - Timpul de așteptare prea mare până la un răspuns
 - Lipsa de informații despre pisici (vârstă, comportament, rasă etc.)
 - Formulare greu de înțeles sau de accesat
 - Faptul că trebuie să merg fizic la adăpost pentru tot
 - Lipsa unui sistem clar de urmărire a cererii
 - Cerințe birocratice sau prea multe acte
 - Comunicarea greoaie cu voluntarii/adăpostul
4. Dacă ar exista o platformă completă online pentru adopția pisicilor, ce ți-ai dori să conțină?
 - Poze, informații detaliate și actualizate despre pisici
 - Formular de adopție online ușor de completat
 - Posibilitatea de a face programări online
 - Notificări despre statusul cererii
 - Posibilitatea de a primi certificatul de adopție digital

- Chat direct cu voluntari sau adăposturi

5. Crezi că digitalizarea procesului de adopție ar încuraja mai mulți oameni să adopte pisici?

- Da, clar
- Probabil da
- Nu știu / Poate
- Nu prea cred
- Nu, deloc

6. Ce te-ar convinge cel mai mult să adopți o pisică de pe o platformă online?

- Posibilitatea de a vedea poze și descrieri clare ale pisicii
- Un proces clar și rapid de aplicare
- Comunicare ușoară cu adăpostul
- Transparența cu privire la istoricul medical și comportament
- Recenzii sau testimoniale ale altor adoptatori

7. Ai încredere să trimiți o cerere de adopție printr-un site online?

- Da, dacă pare profesional și sigur
- Da, dar doar dacă este legat de un ONG cunoscut
- Nu prea, prefer contactul direct
- Nu, nu aș avea încredere